

A Hierarquia Acústica do Afeto: convergência Estrutural Entre Inteligência Artificial e Percepção Humana na Música

Frederico Pedrosa

2026-07-30

Contents

1	CARREGAMENTO E PREPARAÇÃO DOS DADOS	2
1.1	Visualização dos dados brutos em quadrantes	3
1.2	Tentativa por MDS	5
2	ANÁLISE 1: ESTRUTURA LATENTE DO MERT	9
2.1	Análise Paralela de Hoen	9
2.2	Análise Exploratória de Grafos	9
2.3	PCA	12
3	ANÁLISE 2: ESTRUTURA LATENTE HUMANA (A PERCEPÇÃO)	16
3.1	Análise Paralela com os Escores de Valência e Arousal	16
3.2	PCA dados humanos	16
3.3	EGA refinando por estabilidade	18
4	ANÁLISE 3: MODELOS PLS-SEM (MECANISMO DA INTENSIDADE)	24
4.1	Modelo PLS-SEM da Intensidade Acústica	24
4.2	Modelo Humano Hierárquico (PLS)	29
5	Relações entre as variáveis	34
5.1	Diferenças entre o vetor positivo e o negativo	34
6	Russell vs HMA	41

```
if(!require(pacman)) install.packages("pacman")
```

```
## Carregando pacotes exigidos: pacman
```

```
pacman::p_load(dplyr, tidyr, ggplot2, ggrepel, readxl, ggpubr, performance,
               EGAnet, lme4, sjPlot, psych, seminr, boot, stringr, tibble,
               gridExtra, ggpubr, ggeffects, rstatix)
```

1 CARREGAMENTO E PREPARAÇÃO DOS DADOS

```
# --- 1.1 Dados Humanos ---
raw_data <- read_excel("~/PIBIT2025/Circumplex/BasesDados/Marilia/RawData/Raw_Data.xlsx")

# Limpeza e Transformação para Long Format
stimuli_ids <- sprintf("%03d", 1:116)
names(raw_data)[27:142] <- paste0("Emotion_St_", stimuli_ids)
names(raw_data)[144:259] <- paste0("Congruence_St_", stimuli_ids)
names(raw_data)[267:382] <- paste0("Arousal_St_", stimuli_ids)
names(raw_data)[384:499] <- paste0("Valence_St_", stimuli_ids)

# Remover colunas vazias
raw_data <- raw_data[, -c(26, 143, 266, 383)]

# Pivotar (Transformar em Longo)
data_long <- raw_data %>%
  select(Participants, starts_with(c("Emotion_", "Congruence_", "Arousal_", "Valence_"))) %>%
  pivot_longer(
    cols = -Participants,
    names_to = c(".value", "Stimulus_ID"),
    names_pattern = "(.*)_St_(\\d+)"
  ) %>%
  mutate(Stimulus_ID = as.numeric(Stimulus_ID)) %>%
  # Adicionar Emoção Pretendida (Gabarito)
  mutate(Intended_Emotion = case_when(
    Stimulus_ID <= 29 ~ "Happiness",
    Stimulus_ID <= 58 ~ "Fear/Anger",
    Stimulus_ID <= 87 ~ "Serenity",
    TRUE ~ "Sadness"
  ))

# Lista dos 38 Estímulos Validados
ids_38 <- c(4, 33, 45, 61, 64, 66, 71, 81, 86, 110, # AL
            21, 27, 43, 51, 57, 65, 67, 73, 82, 101, # MR
            39, 56, 62, 69, 77, 80, 97, 109, # SE
            25, 42, 44, 50, 54, 76, 79, 83, 95, 106) # TR

# --- 1.2 Dados da IA (MERT) ---
# Carregar Embeddings (Ajuste caminho)
mert_raw <- read_csv("~/PIBIT2025/Circumplex/BasesDados/Marilia/artigo/embeddings_mert_38_musicas.csv",
                    merge_col = "id")
mert_data <- as.data.frame(t(mert_raw)) # Transpor: Linhas=Músicas

# Renomear Músicas para bater com IDs
# Ex: "Alegria - Estímulo 4" -> "AL_4"
clean_names <- colnames(mert_data) # Pegar nomes originais
clean_names <- gsub("Medo Raiva- Estímulo ", "MR_", clean_names)
```

```

clean_names <- gsub("Medo Raiva - Estímulo ", "MR_", clean_names) # ajuste para espaços extras
clean_names <- gsub("Alegria - Estímulo ", "AL_", clean_names)
clean_names <- gsub("Serenidade - Estímulo ", "SE_", clean_names)
clean_names <- gsub("Tristeza - Estímulo ", "TR_", clean_names)
clean_names <- gsub(".mp3", "", clean_names)
colnames(mert_data) <- clean_names

# --- 1.3 Dados Âncora (para PLS-SEM) ---
ancora_raw <- read.csv("~/PIBIT2025/Circumplex/BasesDados/Marilia/artigo/embeddings_mert_ancora.csv", r
ancora_t <- as.data.frame(t(ancora_raw))
colnames(ancora_t) <- c("Ancora_1", "Ancora_2", "Ancora_3", "Ancora_4")

# Unir IA + Âncora para PLS
dados_ia_pls <- cbind(mert_data, ancora_t)

```

1.1 Visualização dos dados brutos em quadrantes

```

# --- 1. Preparar os Dados Agregados ---

# Calcular a média de Valência e Arousal para CADA MÚSICA
humanos_por_musica <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  group_by(Stimulus_ID, Intended_Emotion) %>%
  summarise(
    Mean_Valence = mean(Valence, na.rm = TRUE),
    Mean_Arousal = mean(Arousal, na.rm = TRUE),
    .groups = "drop"
  )

# Segundo: Calcular os CENTRÓIDES (Média das músicas por Categoria)
humanos_centroides <- humanos_por_musica %>%
  group_by(Intended_Emotion) %>%
  summarise(
    Centroid_Val = mean(Mean_Valence),
    Centroid_Aro = mean(Mean_Arousal),
    SD_Val = sd(Mean_Valence), # Desvio padrão (opcional, para ver a dispersão)
    SD_Aro = sd(Mean_Arousal)
  )

# Juntar para poder desenhar as linhas (teias) no gráfico
plot_data_humanos <- left_join(humanos_por_musica, humanos_centroides, by = "Intended_Emotion")

# --- 2. Visualização do Espaço Afetivo Humano ---

ggplot(plot_data_humanos) +
  # Linhas de referência (Centro da escala SAM é 5, assumindo escala 1-9)
  # Se sua escala for -4 a +4, mude para 0. Se for 1 a 9, o centro é 5.
  geom_vline(xintercept = 5, linetype = "dashed", color = "gray70") +
  geom_hline(yintercept = 5, linetype = "dashed", color = "gray70") +

  # Teias (ligando cada música ao seu centróide)

```

```

geom_segment(aes(x = Mean_Valence, y = Mean_Arousal,
                 xend = Centroid_Val, yend = Centroid_Aro,
                 color = Intended_Emotion), alpha = 0.3) +

# Pontos das Músicas Individuais (menores)
geom_point(aes(x = Mean_Valence, y = Mean_Arousal, color = Intended_Emotion),
           size = 3, alpha = 0.6) +

# Pontos dos Centróides (maiores e com borda)
geom_point(data = humanos_centroides,
           aes(x = Centroid_Val, y = Centroid_Aro, fill = Intended_Emotion),
           size = 6, shape = 21, color = "black", stroke = 1.5) +

# Rótulos (Opcional: IDs das músicas)
geom_text_repel(aes(x = Mean_Valence, y = Mean_Arousal, label = Stimulus_ID, color = Intended_Emotion),
               size = 3, max.overlaps = 20) +

# Cores (Mantendo a paleta do paper)
scale_color_manual(values = c("Fear/Anger" = "#D55E00",
                              "Happiness" = "#E69F00",
                              "Sadness" = "#56B4E9",
                              "Serenity" = "#009E73")) +
scale_fill_manual(values = c("Fear/Anger" = "#D55E00",
                              "Happiness" = "#E69F00",
                              "Sadness" = "#56B4E9",
                              "Serenity" = "#009E73")) +

# Ajuste de Escala (SAM vai de 1 a 9)
scale_x_continuous(limits = c(1, 9), breaks = 1:9) +
scale_y_continuous(limits = c(1, 9), breaks = 1:9) +

# Títulos
labs(
  title = "O Espaço Afetivo Percebido (Humanos)",
  subtitle = "Médias de Valência e Arousal (SAM) para os 38 estímulos e seus Centróides",
  x = "Valência Percebida (1 = Negativo, 9 = Positivo)",
  y = "Arousal Percebido (1 = Calmo, 9 = Agitado)",
  color = "Emoção Pretendida",
  fill = "Emoção Pretendida"
) +
theme_minimal(base_size = 14) +
theme(panel.grid.minor = element_blank())

```

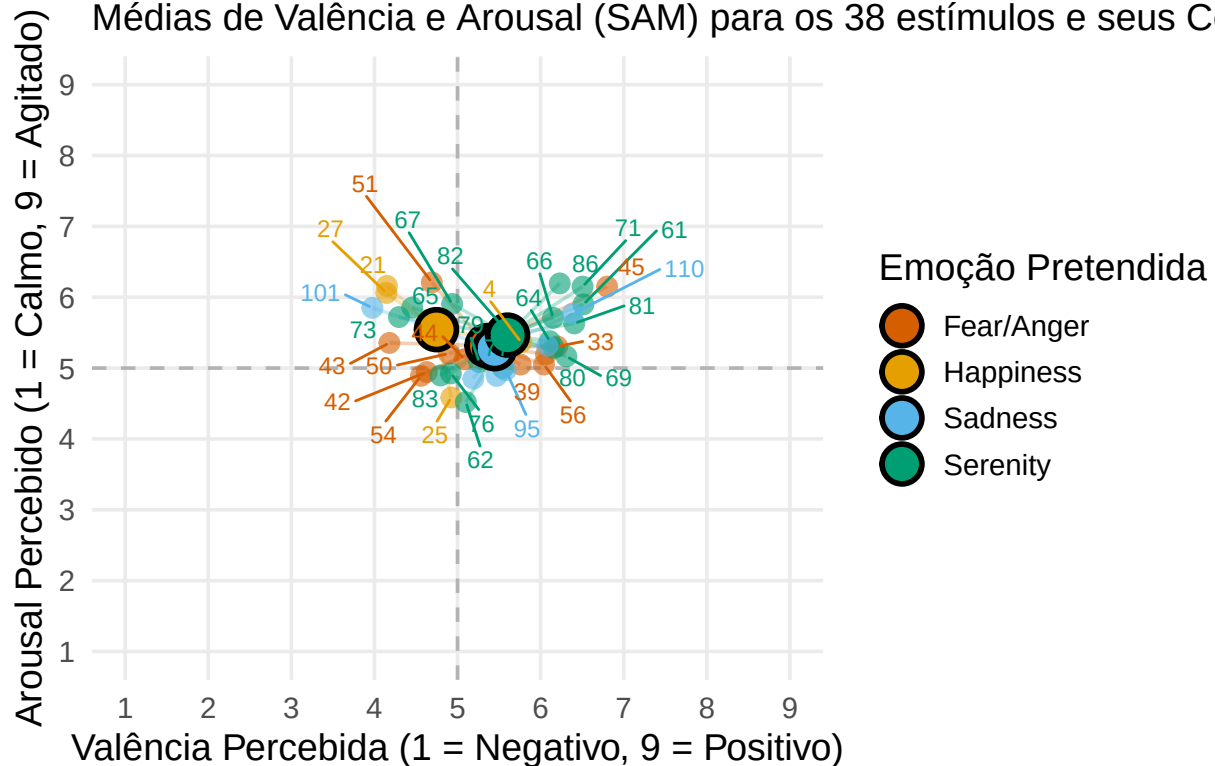
```

## Warning: ggrepel: 4 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps

```

O Espaço Afetivo Percebido (Humanos)

Médias de Valência e Arousal (SAM) para os 38 estímulos e seus Cer



1.2 Tentativa por MDS

```
# 1. Preparar os dados (Médias por música)
humanos_mds_data <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  group_by(Stimulus_ID, Intended_Emotion) %>%
  summarise(
    Val = mean(Valence, na.rm = TRUE),
    Aro = mean(Arousal, na.rm = TRUE),
    .groups = "drop"
  )

# 2. Calcular Matriz de Distância
# "O quão distante a música A está da música B na mente dos participantes?"
dist_matrix <- dist(humanos_mds_data[, c("Val", "Aro")])

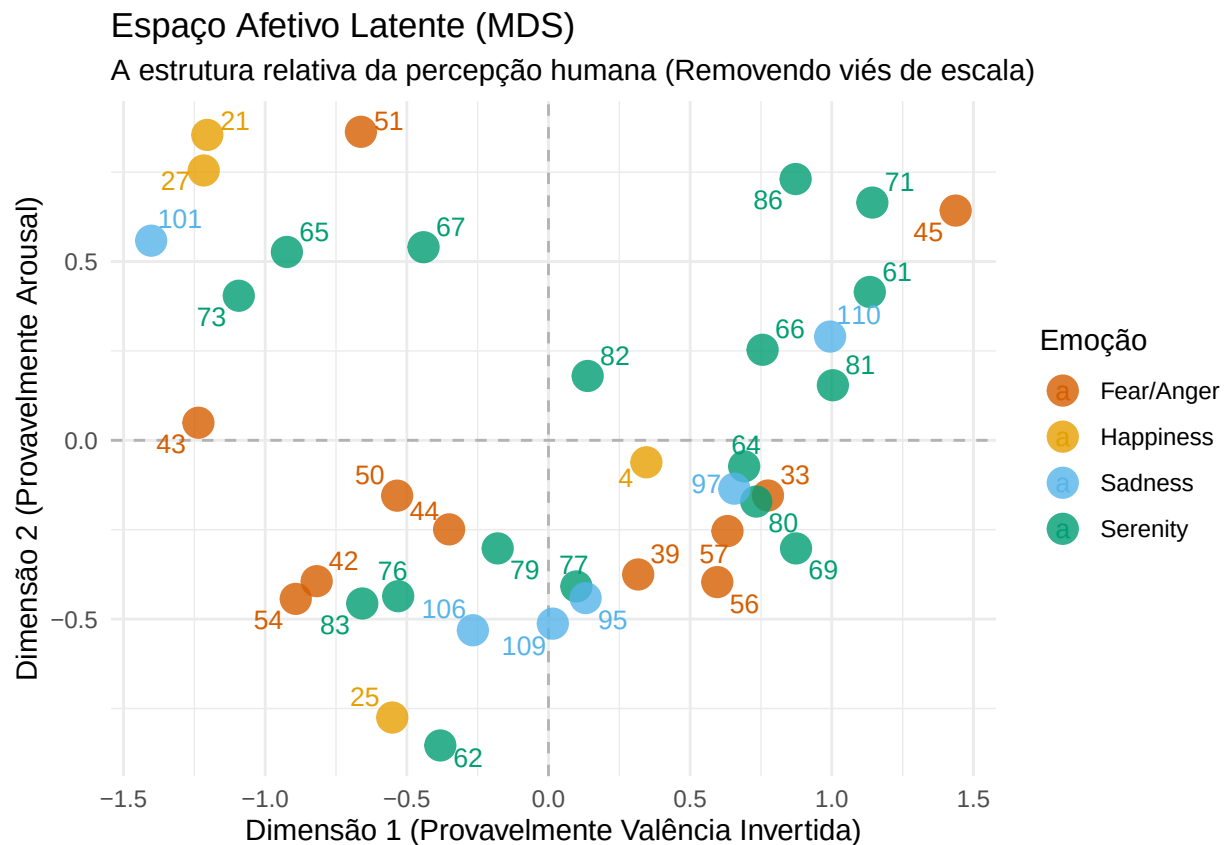
# 3. Rodar MDS (Multidimensional Scaling)
fit_mds <- cmdscale(dist_matrix, eig = TRUE, k = 2)
df_mds <- data.frame(
  Dim1 = fit_mds$points[, 1],
  Dim2 = fit_mds$points[, 2],
  Stimulus_ID = humanos_mds_data$Stimulus_ID,
  Emoção = humanos_mds_data$Intended_Emotion
)
```

```
# 4. Plotar o MDS
ggplot(df_mds, aes(x = Dim1, y = Dim2, color = Emoção)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray70") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray70") +

  geom_point(size = 5, alpha = 0.8) +
  geom_text_repel(aes(label = Stimulus_ID), size = 3.5, max.overlaps = 20) +

  scale_color_manual(values = c("Fear/Anger" = "#D55E00",
                                "Happiness" = "#E69F00",
                                "Sadness" = "#56B4E9",
                                "Serenity" = "#009E73")) +

  labs(
    title = "Espaço Afetivo Latente (MDS)",
    subtitle = "A estrutura relativa da percepção humana (Removendo viés de escala)",
    x = "Dimensão 1 (Provavelmente Valência Invertida)",
    y = "Dimensão 2 (Provavelmente Arousal)"
  ) +
  theme_minimal()
```



1.2.1 Sóciodemográficos

```
# 2. Análise de Sexo (Frequência e Porcentagem)
sexo_tabela <- table(raw_data$Sex)
print(sexo_tabela)
```

```
##
##      1      2
## 104    96
```

```
prop.table(sexo_tabela)
```

```
##
##      1      2
## 0.52 0.48
```

```
# 3. Descritivas por Grupo (Média, DP, Mediana, Skew, etc.)
# Idade
describeBy(raw_data$Age, group = raw_data$Group)
```

```
##
## Descriptive statistics by group
## group: 1
##      vars      n mean    sd median trimmed  mad min max range skew kurtosis  se
## X1      1 100 27.14 5.65     26    26.8 5.93  18  40    22 0.44   -0.76 0.57
## -----
## group: 2
##      vars      n mean    sd median trimmed  mad min max range skew kurtosis  se
## X1      1 100 27.4 6.17     27    27 7.41  19  43    24 0.47   -0.94 0.62
```

```
# Escolaridade (se for categórica, o ideal é table; se for ordinal, describeBy)
describeBy(raw_data$Education, group = raw_data$Group)
```

```
##
## Descriptive statistics by group
## group: 1
##      vars      n mean    sd median trimmed  mad min max range skew kurtosis  se
## X1      1 100   1.5 0.5     1.5    1.5 0.74   1   2     1   0   -2.02 0.05
## -----
## group: 2
##      vars      n mean    sd median trimmed  mad min max range skew kurtosis  se
## X1      1 100   1.5 0.5     1.5    1.5 0.74   1   2     1   0   -2.02 0.05
```

```
# Horas de Estudo Musical (Atenção às crases devido aos espaços no nome da coluna)
describeBy(raw_data$`Hours of musical study`, group = raw_data$Group)
```

```
##
## Descriptive statistics by group
## group: 1
```

```
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100 14.91 11.73     10   13.59 10.38    0 60   60 1.09      1.15 1.17
## -----
## group: 2
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100  0.12  0.92      0      0    0    0  9    9 8.96     82.41 0.09
```

```
# 4. Indicadores Clínicos (BAI e BDI) - Descritivas
```

```
describeBy(raw_data$BAI, group = raw_data$Group)
```

```
##
## Descriptive statistics by group
## group: 1
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100  6.69  5.95      6   5.82 4.45    0 29   29 1.53      2.78 0.6
## -----
## group: 2
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100  8.77  7.22      7   7.82 5.93    0 30   30 1.05      0.5 0.72
```

```
describeBy(raw_data$BDI, group = raw_data$Group)
```

```
##
## Descriptive statistics by group
## group: 1
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100  7.93  6.1      7   7.32 5.93    0 26   26 0.8      0.06 0.61
## -----
## group: 2
##      vars   n mean    sd median trimmed   mad min max range skew kurtosis   se
## X1      1 100  8.11  5.86      8   7.55 5.93    0 30   30 1.16      2 0.59
```

```
# 5. Testes de Comparação (Wilcoxon - Não Paramétrico)
```

```
# Testando Ansiedade (BAI)
```

```
wilcox_bai <- wilcox.test(BAI ~ Group, data = raw_data, exact = FALSE)
print(wilcox_bai)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
## data: BAI by Group
## W = 4167, p-value = 0.04144
## alternative hypothesis: true location shift is not equal to 0
```

```
# Testando Depressão (BDI)
```

```
wilcox_bdi <- wilcox.test(BDI ~ Group, data = raw_data, exact = FALSE)
print(wilcox_bdi)
```

```
##
## Wilcoxon rank sum test with continuity correction
##
```



```
## data: BDI by Group
## W = 4839, p-value = 0.6944
## alternative hypothesis: true location shift is not equal to 0
```

```
# 6. Tamanho do Efeito (r de Cohen para Wilcoxon)
# Essencial para reportar o p=0.041 do BAI
eff_bai <- wilcox_effsize(raw_data, BAI ~ Group)
print(eff_bai)
```

```
## # A tibble: 1 x 7
##   .y. group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr> <chr>    <dbl> <int> <int> <ord>
## 1 BAI    1      2      0.144   100   100 small
```

```
eff_bdi <- wilcox_effsize(raw_data, BDI ~ Group)
print(eff_bdi)
```

```
## # A tibble: 1 x 7
##   .y. group1 group2 effsize    n1    n2 magnitude
## * <chr> <chr> <chr>    <dbl> <int> <int> <ord>
## 1 BDI    1      2      0.0279   100   100 small
```

2 ANÁLISE 1: ESTRUTURA LATENTE DO MERT

2.1 Análise Paralela de Hoen

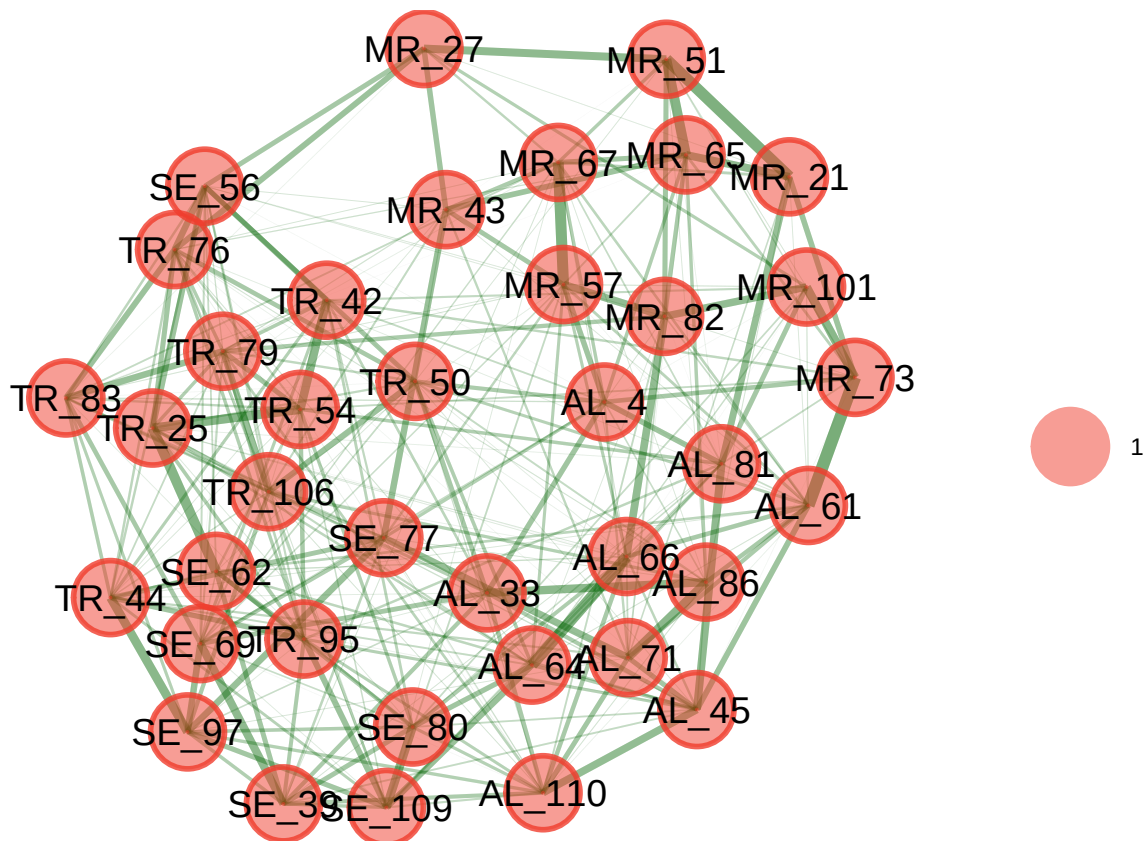
```
# Análise Paralela dos Embeddings
resultado_paralela <- fa.parallel(mert_data, fa = "both", n.iter = 100,
                                main = "Análise Paralela de Horn (MERT Embeddings)",
                                plot = F)
```

```
## Parallel analysis suggests that the number of factors = 2 and the number of components = 1
```

```
# Exibir os valores numéricos
#print(resultado_paralela)
```

2.2 Análise Exploratória de Grafos

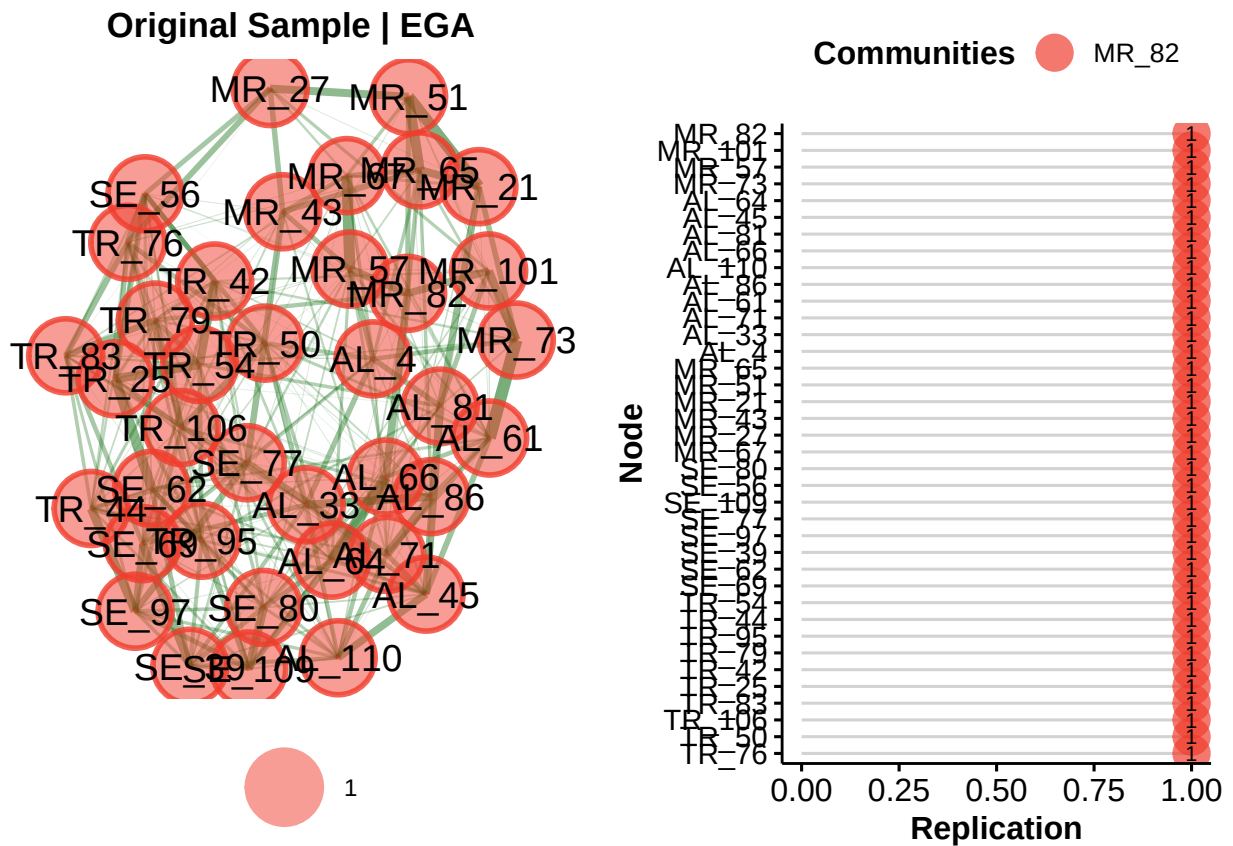
```
# EGA dos Embeddings médios
EGA(mert_data)
```



```
## Model: GLASSO (EBIC with gamma = 0.5)
## Correlations: auto
## Lambda: 0.0975579351612073 (n = 100, ratio = 0.1)
##
## Number of nodes: 38
## Number of edges: 288
## Edge density: 0.410
##
## Non-zero edge weights:
##      M      SD      Min      Max
## 0.065 0.053 0.000 0.270
##
## ----
##
## Algorithm: Louvain
##
## Number of communities: 1
##
## MR_82 MR_101 MR_57 MR_73 AL_64 AL_45 AL_81 AL_66 AL_110 AL_86 AL_61
##      1      1      1      1      1      1      1      1      1      1      1
## AL_71 AL_33 AL_4 MR_65 MR_51 MR_21 MR_43 MR_27 MR_67 SE_80 SE_56
##      1      1      1      1      1      1      1      1      1      1      1
## SE_109 SE_77 SE_97 SE_39 SE_62 SE_69 TR_54 TR_44 TR_95 TR_79 TR_42
##      1      1      1      1      1      1      1      1      1      1      1
## TR_25 TR_83 TR_106 TR_50 TR_76
##      1      1      1      1      1
```

```
##
## ----
##
## Unidimensional Method: Louvain
## Unidimensional: Yes
##
## ----
##
## TEFI: 0
```

```
boot_IA <- bootEGA(mert_data, seed = 123)
```



```
#png("Figura_2.png", width = 3000, height = 1600, res = 300)

# Gera o gráfico (o EGAnet usa ggplot internamente)
#plot(boot_IA,
#      node.size = 6,
#      label.size = 3)

# Fecha e salva o arquivo
#dev.off()
```

2.3 PCA

```
pca_ia <- principal(mert_data, nfactors = 3, rotate = "none")
print("--- IA: Variância Explicada (PC1 = Intensidade, PC2 = Valência, PC3 = Ativação) ---")

## [1] "--- IA: Variância Explicada (PC1 = Intensidade, PC2 = Valência, PC3 = Ativação) ---"

print(pca_ia$Vaccounted[2,])

##          PC1          PC2          PC3
## 0.87694180 0.02287581 0.01361539

pca_ia <- prcomp(mert_data, scale. = TRUE)
pca_ia$rotation[1:38] # primeiro Componente todos loadings positivos

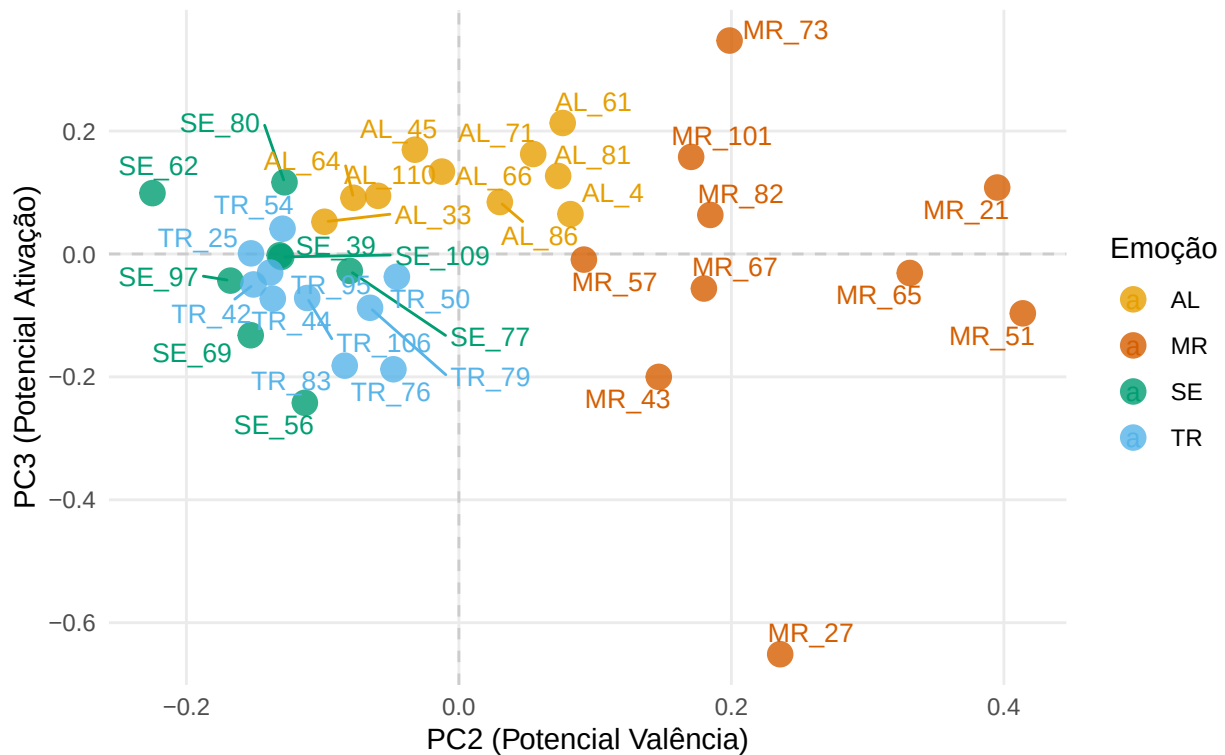
## [1] 0.1612043 0.1478440 0.1654314 0.1558623 0.1660356 0.1682248 0.1648897
## [8] 0.1656165 0.1679097 0.1595525 0.1660760 0.1618144 0.1649603 0.1644037
## [15] 0.1556717 0.1479055 0.1478102 0.1531097 0.1387216 0.1611846 0.1677300
## [22] 0.1622147 0.1673056 0.1655734 0.1662498 0.1648651 0.1617196 0.1658004
## [29] 0.1645854 0.1664473 0.1678262 0.1676180 0.1610400 0.1658287 0.1630971
## [36] 0.1658952 0.1653830 0.1656900

# Gráfico do Espaço Latente (A Hierarquia)
df_pca_residuo <- data.frame(
  Coord_PC2 = pca_ia$rotation[,2],
  Coord_PC3 = pca_ia$rotation[,3],
  Nome = rownames(pca_ia$rotation),
  Categoria = substr(rownames(pca_ia$rotation), 1, 2)
)

ggplot(df_pca_residuo, aes(x = Coord_PC2, y = Coord_PC3, color = Categoria)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray80") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray80") +
  geom_point(size = 4, alpha = 0.8) +
  geom_text_repel(aes(label = Nome), size = 3.5, max.overlaps = 30) +
  scale_color_manual(values = c("AL" = "#E69F00", "MR" = "#D55E00",
                                "SE" = "#009E73", "TR" = "#56B4E9")) +
  labs(
    title = "O 'Circumplexo Residual' da IA (MERT)",
    subtitle = "Espaço qualitativo após remover a Intensidade (PC1)",
    x = "PC2 (Potencial Valência)",
    y = "PC3 (Potencial Ativação)",
    color = "Emoção"
  ) +
  theme_minimal() +
  theme(panel.grid.minor = element_blank())
```

O 'Circumplexo Residual' da IA (MERT)

Espaço qualitativo após remover a Intensidade (PC1)



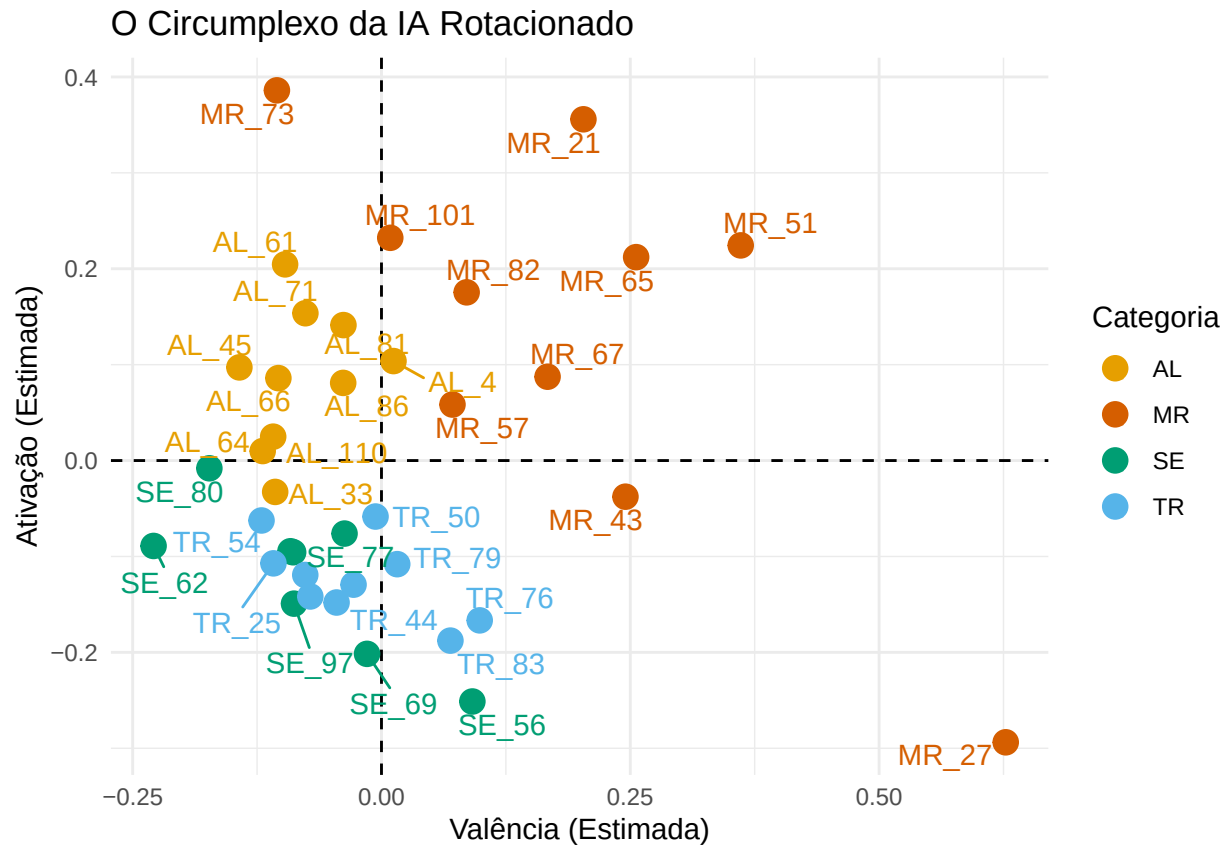
2.3.1 Rotacionar 45 graus para verificar o circumplexo

```
# Ângulo de rotação (ajuste conforme necessário, comecei com 45 graus)
angulo <- 45 * (pi / 180)

df_pca_residuo$PC2_rot <- df_pca_residuo$Coord_PC2 * cos(angulo) - df_pca_residuo$Coord_PC3 * sin(angulo)
df_pca_residuo$PC3_rot <- df_pca_residuo$Coord_PC2 * sin(angulo) + df_pca_residuo$Coord_PC3 * cos(angulo)

# Plot rotacionado
ggplot(df_pca_residuo, aes(x = PC2_rot, y = PC3_rot, color = Categoria)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_point(size = 4) +
  geom_text_repel(aes(label = Nome)) +
  scale_color_manual(values = c("AL"="#E69F00", "MR"="#D55E00", "SE"="#009E73", "TR"="#56B4E9")) +
  labs(title = "O Circumplexo da IA Rotacionado", x = "Valência (Estimada)", y = "Ativação (Estimada)") +
  theme_minimal()

## Warning: ggrepel: 5 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



2.3.2 Além de rotacionado, estimando os centróides

```
# 1. Calcular os Centróides (Média de X e Y por Categoria)
centroides <- df_pca_residuo %>%
  group_by(Categoria) %>%
  summarise(
    PC2_cent = mean(PC2_rot),
    PC3_cent = mean(PC3_rot)
  )

# 2. Gerar o gráfico com os Centros de Gravidade
ggplot() +
  # Linhas de referência
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray80") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray80") +

  # Desenha "teias" ligando os pontos ao centro (opcional, mas fica lindo)
  geom_segment(data = df_pca_residuo %>% left_join(centroides, by="Categoria"),
    aes(x = PC2_rot, y = PC3_rot, xend = PC2_cent, yend = PC3_cent, color = Categoria),
    alpha = 0.3) +

  # Pontos das músicas individuais
  geom_point(data = df_pca_residuo, aes(x = PC2_rot, y = PC3_rot, color = Categoria),
    size = 3, alpha = 0.6) +
```

```

# OS CENTRÓIDES (Pontos maiores e com borda)
geom_point(data = centroides, aes(x = PC2_cent, y = PC3_cent, fill = Categoria),
           size = 6, shape = 21, color = "black", stroke = 1.5) +

# Rótulos dos estímulos
geom_text_repel(data = df_pca_residuo, aes(x = PC2_rot, y = PC3_rot, label = Nome, color = Categoria),
               size = 3) +

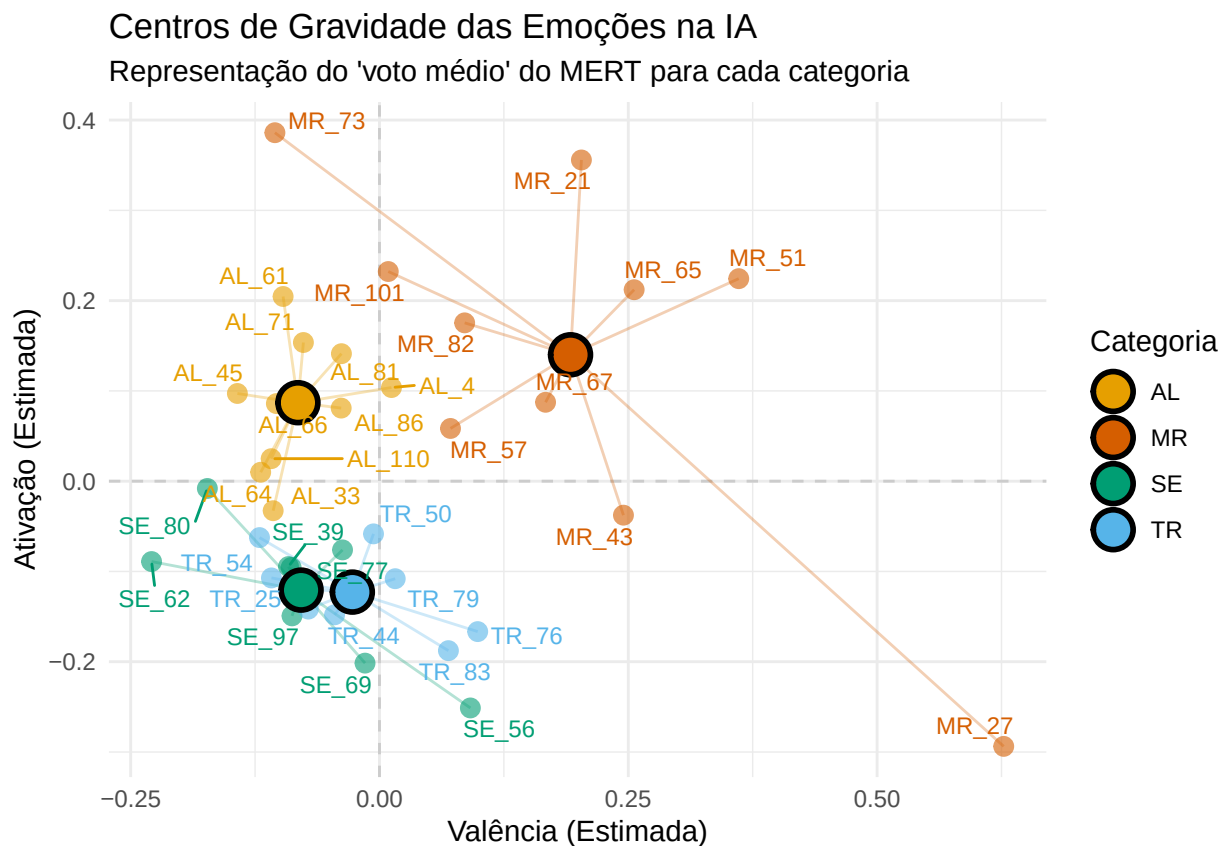
# Estética e Cores
scale_color_manual(values = c("AL"="#E69F00", "MR"="#D55E00", "SE"="#009E73", "TR"="#56B4E9")) +
scale_fill_manual(values = c("AL"="#E69F00", "MR"="#D55E00", "SE"="#009E73", "TR"="#56B4E9")) +
labs(
  title = "Centros de Gravidade das Emoções na IA",
  subtitle = "Representação do 'voto médio' do MERT para cada categoria",
  x = "Valência (Estimada)",
  y = "Ativação (Estimada)"
) +
theme_minimal()

```

```

## Warning: ggrepel: 4 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps

```



3 ANÁLISE 2: ESTRUTURA LATENTE HUMANA (A PERCEPÇÃO)

3.1 Análise Paralela com os Escores de Valência e Arousal

```
# 1. Preparar Matriz Wide Completa (76 colunas)
human_wide_total <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  # Criar uma coluna única para Identificador e Atributo
  pivot_longer(cols = c(Arousal, Valence), names_to = "Attribute", values_to = "Score") %>%
  mutate(Var_Name = paste0("St", Stimulus_ID, "_", Attribute)) %>%
  select(Participants, Var_Name, Score) %>%
  pivot_wider(names_from = Var_Name, values_from = Score) %>%
  na.omit()

# 2. Análise Paralela de Horn (Quantas dimensões existem no espaço total?)
pa_total <- fa.parallel(human_wide_total[-1], fa = "both", n.iter = 100,
  main = "", plot = F)
```

```
## Parallel analysis suggests that the number of factors = 5 and the number of components = 5
```

3.2 PCA dados humanos

```
# 4. PCA Hierárquico (O teste da Intensidade como Fator Geral)
pca_hum <- principal(human_wide_total[-1], nfactors = 5, rotate = "none")
pca_hum$loadings
```

```
##
## Loadings:
##
```

	PC1	PC2	PC3	PC4	PC5
## St4_Arousal	0.607		-0.125		0.140
## St4_Valence	0.336	0.473		-0.306	0.134
## St21_Arousal	0.464	-0.287	0.198		0.416
## St21_Valence	0.140	0.469	-0.120	0.302	
## St25_Arousal	0.630		-0.312	-0.292	
## St25_Valence	0.329	0.543	-0.138		
## St27_Arousal	0.535	-0.296			0.271
## St27_Valence	0.189	0.476	-0.299	0.388	
## St33_Arousal	0.623	-0.301			0.218
## St33_Valence	0.337	0.153	0.377	-0.208	0.153
## St39_Arousal	0.619	-0.145	-0.238	-0.141	-0.191
## St39_Valence	0.309	0.327	0.354	-0.194	
## St42_Arousal	0.553	-0.116	-0.424	-0.181	-0.194
## St42_Valence	0.280	0.432	-0.156		
## St43_Arousal	0.517	-0.356			0.181
## St43_Valence	0.226	0.473	-0.147	0.300	
## St44_Arousal	0.655	-0.169	-0.240		0.173
## St44_Valence	0.266	0.619	-0.166	0.153	
## St45_Arousal	0.574	-0.195	0.228	0.289	

## St45_Valence	0.348		0.628		-0.194
## St50_Arousal	0.632	-0.159	-0.274	-0.179	0.231
## St50_Valence	0.263	0.377	0.218	-0.197	0.267
## St51_Arousal	0.479	-0.277	0.157	0.252	0.218
## St51_Valence	0.227	0.583	-0.195	0.206	
## St54_Arousal	0.622	-0.122	-0.145		-0.122
## St54_Valence	0.301	0.469		0.133	
## St56_Arousal	0.600	-0.194	-0.175		-0.170
## St56_Valence	0.397	0.297	0.363		-0.290
## St57_Arousal	0.566	-0.254		0.250	-0.114
## St57_Valence	0.364	0.276	0.314	0.139	-0.222
## St61_Arousal	0.644	-0.249	0.142	0.343	
## St61_Valence	0.321		0.556		
## St62_Arousal	0.589		-0.403		-0.203
## St62_Valence	0.366	0.490			
## St64_Arousal	0.635	-0.251	-0.191	-0.136	0.155
## St64_Valence	0.365	0.210	0.295	-0.317	0.270
## St65_Arousal	0.564	-0.333	0.172	0.219	0.348
## St65_Valence	0.124	0.514	-0.178	0.158	0.276
## St66_Arousal	0.660	-0.243		0.277	-0.132
## St66_Valence	0.364		0.476		-0.204
## St67_Arousal	0.639	-0.270	0.159	0.216	0.272
## St67_Valence	0.306	0.526	-0.154	0.134	0.196
## St69_Arousal	0.625	-0.172	-0.303	-0.150	0.156
## St69_Valence	0.415	0.360	0.308	-0.119	0.172
## St71_Arousal	0.660	-0.252	0.114	0.232	
## St71_Valence	0.414		0.431	-0.211	
## St73_Arousal	0.595	-0.265	0.115	0.205	0.107
## St73_Valence	0.308	0.536	-0.213	0.176	0.130
## St76_Arousal	0.718		-0.219	-0.137	
## St76_Valence	0.321	0.649	-0.128	0.140	
## St77_Arousal	0.665	-0.159	-0.174	-0.284	
## St77_Valence	0.343	0.239	0.263	-0.369	0.138
## St79_Arousal	0.696		-0.217		-0.217
## St79_Valence	0.362	0.350		0.234	-0.180
## St80_Arousal	0.609		-0.366	-0.185	-0.237
## St80_Valence	0.304	0.405	0.121	-0.263	
## St81_Arousal	0.679	-0.216		0.181	
## St81_Valence	0.393	0.248	0.366	-0.145	
## St82_Arousal	0.631	-0.159			0.308
## St82_Valence	0.333	0.504			
## St83_Arousal	0.686	-0.129	-0.170	-0.208	-0.177
## St83_Valence	0.378	0.572	-0.128	-0.103	
## St86_Arousal	0.618	-0.195	0.197	0.312	
## St86_Valence	0.348		0.562	-0.127	-0.160
## St95_Arousal	0.698		-0.294	-0.172	
## St95_Valence	0.387	0.498	0.140		0.150
## St97_Arousal	0.622	-0.149	-0.229		-0.411
## St97_Valence	0.409	0.210	0.172		-0.435
## St101_Arousal	0.568	-0.382	0.129		
## St101_Valence	0.236	0.524	-0.259	0.313	
## St106_Arousal	0.657		-0.320	-0.246	
## St106_Valence	0.360	0.496	0.189		
## St109_Arousal	0.654		-0.260	-0.218	

```
## St109_Valence 0.385 0.394 0.170 -0.176
## St110_Arousal 0.578 -0.381 0.183 0.322 -0.208
## St110_Valence 0.427 0.577 -0.276
##
##          PC1  PC2  PC3  PC4  PC5
## SS loadings 18.651 8.136 5.018 2.801 2.382
## Proportion Var 0.245 0.107 0.066 0.037 0.031
## Cumulative Var 0.245 0.352 0.418 0.455 0.487
```

```
# 5. Os loadings de Valência e Arousal são ambos positivos no PC1?
# Vamos criar um dataframe para visualizar isso
loadings_total <- as.data.frame(unclass(pca_hum$loadings[,1, drop=FALSE]))
colnames(loadings_total) <- "PC1_Loading"
loadings_total$Variable <- rownames(loadings_total)

# Separar nome da música e atributo
loadings_total <- loadings_total %>%
  separate(Variable, into = c("ID", "Atributo"), sep = "_")

# Ver a média de carga por atributo
loadings_total %>%
  group_by(Atributo) %>%
  summarise(Media_Carga_no_Fator_Geral = mean(PC1_Loading))
```

```
## # A tibble: 2 x 2
##   Atributo Media_Carga_no_Fator_Geral
##   <chr>          <dbl>
## 1 Arousal          0.615
## 2 Valence          0.323
```

3.3 EGA refinando por estabilidade

```
current_data_hum <- human_wide_total[-1]

# Passo 1: Unique Variable Analysis (UVA) não identificou itens redundantes
(uva_results_hum <- UVA(data = current_data_hum, verbose = FALSE))

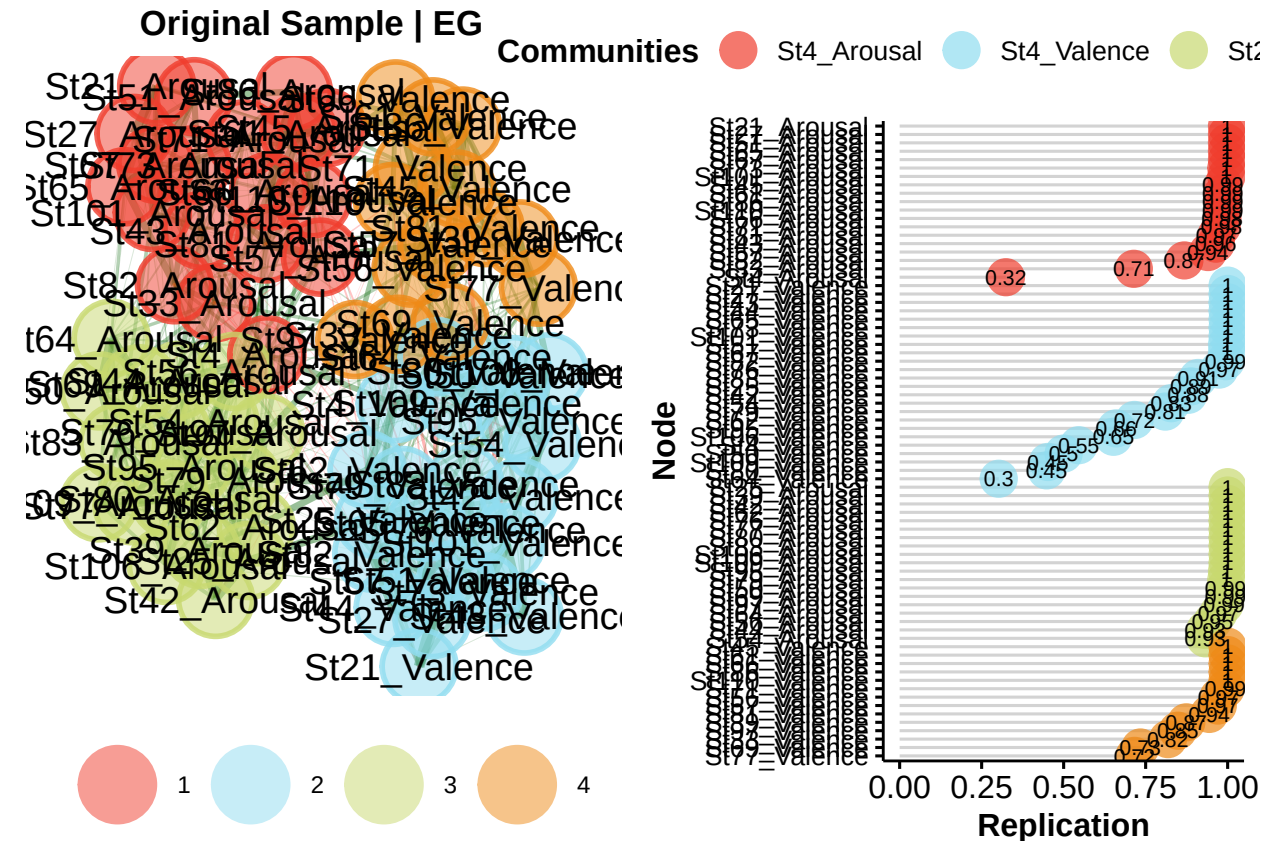
## Variable pairs with wTO > 0.30 (large-to-very large redundancy)
##
## ----
##
## Variable pairs with wTO > 0.25 (moderate-to-large redundancy)
##
## ----
##
## Variable pairs with wTO > 0.20 (small-to-moderate redundancy)

# Passo 2: Bootstrap EGA para calcular a estabilidade dos itens
boot_refine_hum <- bootEGA(
  data = current_data_hum,
  iter = 500,
```

```

type = "parametric",
model = "glasso",
seed = 123
)

```



```

# Passo 3: Extração dos índices de estabilidade de nó
item_stability_hum <- boot_refine_hum$stability$item.stability$item.stability$empirical.dimensions

# Definição do limiar de estabilidade (Threshold padrão de 0.75)
threshold <- 0.75

# Identificação de itens estáveis e instáveis
stable_items_hum <- names(item_stability_hum[item_stability_hum >= threshold])
unstable_items_hum <- names(item_stability_hum[item_stability_hum < threshold])

# Passo 4: Relatório de Refinamento
cat("\n>>> RESULTADOS DO REFINAMENTO <<<\n")

##
## >>> RESULTADOS DO REFINAMENTO <<<

cat("Itens Estáveis (>= 0.75):", length(stable_items_hum), "\n")

## Itens Estáveis (>= 0.75): 64

```

```

cat("Itens Instáveis (< 0.75):", length(unstable_items_hum), "\n")

## Itens Instáveis (< 0.75): 12

if (length(unstable_items_hum) > 0) {
  cat("\nEstímulos indicados para exclusão por instabilidade estrutural:\n")
  print(unstable_items_hum)
}

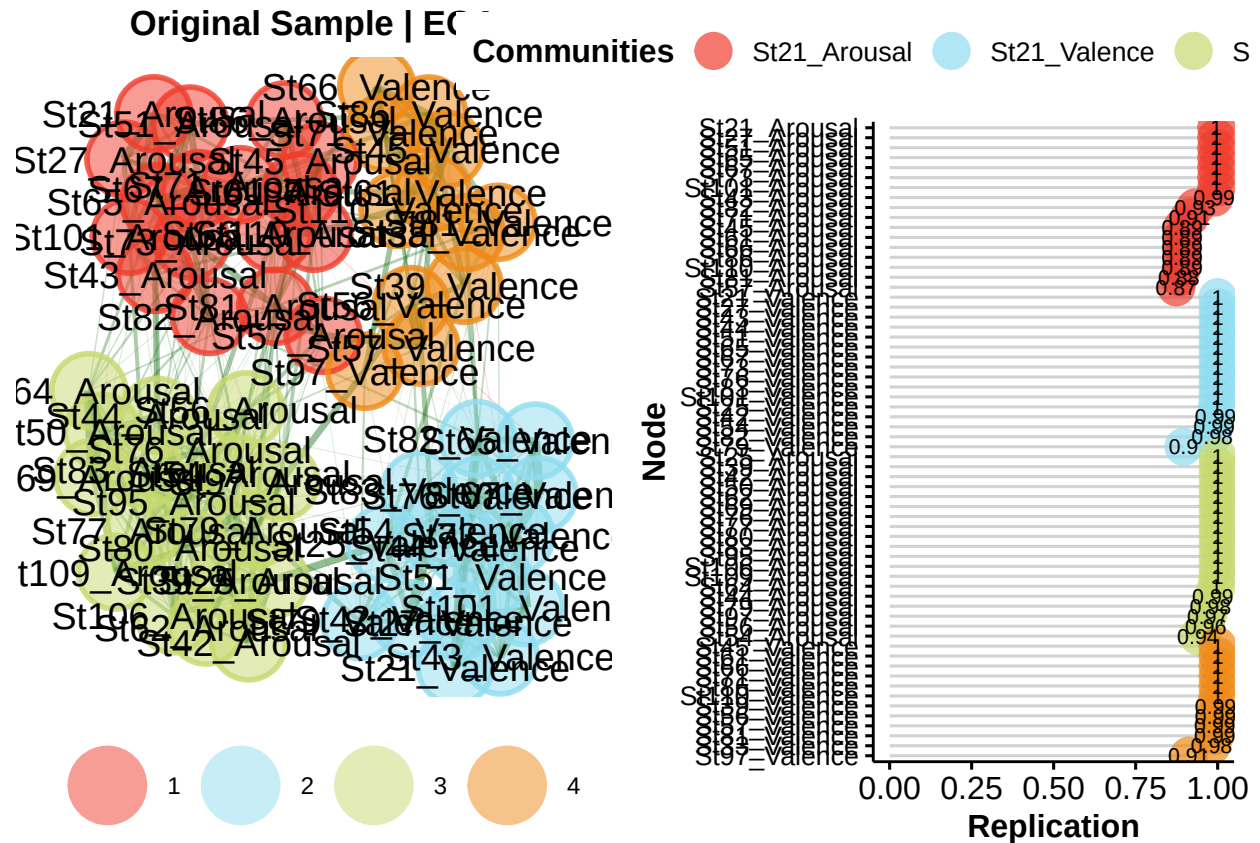
##
## Estímulos indicados para exclusão por instabilidade estrutural:
## [1] "St4_Arousal" "St4_Valence" "St33_Arousal" "St50_Valence"
## [5] "St62_Valence" "St64_Valence" "St69_Valence" "St77_Valence"
## [9] "St80_Valence" "St95_Valence" "St106_Valence" "St109_Valence"

# Passo 5: Criação do dataset refinado para análise final
if (length(stable_items_hum) > 0) {
  human_refined_data <- current_data_hum[, stable_items_hum, drop = FALSE]
  cat("\nDimensões do dataset refinado (Participantes x Estímulos Estáveis):\n")
  print(dim(human_refined_data))

  # Calcular a estrutura final com os itens estáveis
  ega_final_hum <- bootEGA(human_refined_data, model = "glasso", seed = 123)
} else {
  stop("Nenhum item atingiu o limiar de estabilidade na Iteração 1.")
}

##
## Dimensões do dataset refinado (Participantes x Estímulos Estáveis):
## [1] 200 64

```



```
#png("Figura_4.png", width = 3200, height = 2100, res = 300)

# Gera o gráfico com os parâmetros de rede
#plot(ega_final_hum,
#      node.size = 5,
#      label.size = 2.8)

# Fecha e salva o arquivo
#dev.off()
```

3.3.1 Identificação da composição das comunidades

```
# 1. Extrair a alocação de cada item
membership <- ega_final_hum$EGA$wc

# 2. Criar o dataframe corrigindo o Regex para os novos nomes
df_comunidades <- data.frame(
  Item = names(membership),
  Comunidade = as.vector(membership)
) %>%
  # Extrair apenas o número (ID) e o Atributo (Valence/Arousal)
  mutate(
    Stimulus_ID = as.numeric(stringr::str_extract(Item, "\\d+")),
    Atributo = stringr::str_extract(Item, "Valence|Arousal")
  )
```

```

)

# 3. Juntar com o gabarito de categorias
# Criando o gabarito oficial com os 38 estímulos validados no artigo
gabarito_categorias <- data.frame(
  Stimulus_ID = c(
    4, 33, 45, 61, 64, 66, 71, 81, 86, 110, # IDs de Alegria (AL)
    21, 27, 43, 51, 57, 65, 67, 73, 82, 101, # IDs de Medo/Raiva (MR)
    39, 56, 62, 69, 77, 80, 97, 109, # IDs de Serenidade (SE)
    25, 42, 44, 50, 54, 76, 79, 83, 95, 106 # IDs de Tristeza (TR)
  )
) %>%
mutate(Intended_Emotion = case_when(
  Stimulus_ID %in% c(4, 33, 45, 61, 64, 66, 71, 81, 86, 110) ~ "Alegria",
  Stimulus_ID %in% c(21, 27, 43, 51, 57, 65, 67, 73, 82, 101) ~ "Medo/Raiva",
  Stimulus_ID %in% c(39, 56, 62, 69, 77, 80, 97, 109) ~ "Serenidade",
  Stimulus_ID %in% c(25, 42, 44, 50, 54, 76, 79, 83, 95, 106) ~ "Tristeza",
  TRUE ~ "Outro"
))

resultado_final <- df_comunidades %>%
  left_join(gabarito_categorias, by = "Stimulus_ID")

# --- RELATÓRIO DE COMPOSIÇÃO ---
resumo_comunidades <- resultado_final %>%
  group_by(Comunidade, Intended_Emotion, Atributo) %>%
  tally() %>%
  arrange(Comunidade, Atributo)

print(resumo_comunidades)

## # A tibble: 10 x 4
## # Groups:   Comunidade, Intended_Emotion [10]
##   Comunidade Intended_Emotion Atributo      n
##   <dbl> <chr> <chr> <int>
## 1         1 Alegria Arousal      7
## 2         1 Medo/Raiva Arousal     10
## 3         2 Medo/Raiva Valence      9
## 4         2 Tristeza Valence      7
## 5         3 Alegria Arousal      1
## 6         3 Serenidade Arousal      8
## 7         3 Tristeza Arousal     10
## 8         4 Alegria Valence      8
## 9         4 Medo/Raiva Valence      1
## 10        4 Serenidade Valence      3

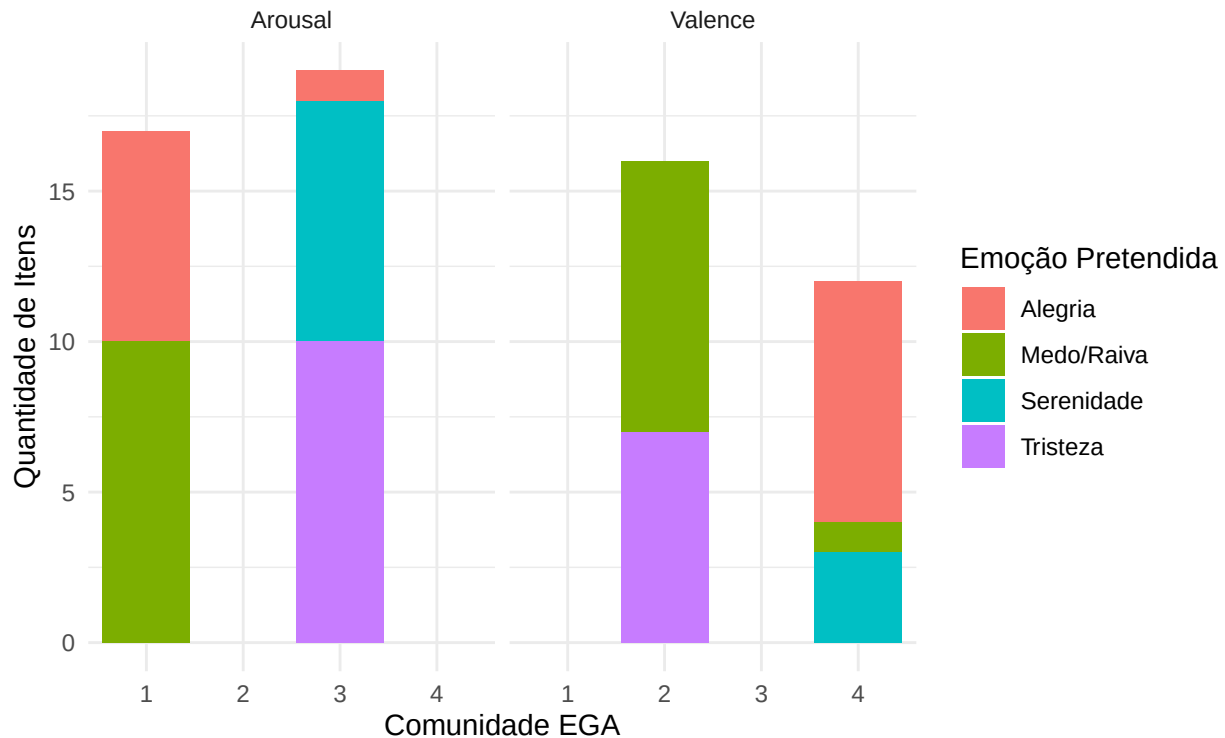
# 4. Visualização da "Arquitetura do Afeto"
ggplot(resultado_final, aes(x = factor(Comunidade), fill = Intended_Emotion)) +
  geom_bar(position = "stack") +
  facet_wrap(~Atributo) +
  labs(title = "As 4 Facetas do Afeto Humano",
       subtitle = "Separação funcional de Valência e Arousal por comunidades de rede",

```

```
x = "Comunidade EGA", y = "Quantidade de Itens",
fill = "Emoção Pretendida") +
theme_minimal()
```

As 4 Facetas do Afeto Humano

Separação funcional de Valência e Arousal por comunidades de rede



```
# 1. Calcular a porcentagem de cada emoção DENTRO de cada comunidade
df_porcentagens <- resultado_final %>%
  group_by(Comunidade) %>%
  mutate(Total_na_Comunidade = n()) %>% # Total de itens (nós) na comunidade
  group_by(Comunidade, Intended_Emotion) %>%
  summarise(
    Quantidade = n(),
    Porcentagem = (n() / first(Total_na_Comunidade)) * 100,
    .groups = "drop"
  )

# 2. Exibir o resultado formatado
print(df_porcentagens %>% arrange(Comunidade, desc(Porcentagem)))
```

```
## # A tibble: 10 x 4
##   Comunidade Intended_Emotion Quantidade Porcentagem
##   <dbl> <chr> <int> <dbl>
## 1 1 Medo/Raiva 10 58.8
## 2 1 Alegria 7 41.2
## 3 2 Medo/Raiva 9 56.2
## 4 2 Tristeza 7 43.8
```

## 5	3 Tristeza	10	52.6
## 6	3 Serenidade	8	42.1
## 7	3 Alegria	1	5.26
## 8	4 Alegria	8	66.7
## 9	4 Serenidade	3	25
## 10	4 Medo/Raiva	1	8.33

4 ANÁLISE 3: MODELOS PLS-SEM (MECANISMO DA INTENSIDADE)

4.1 Modelo PLS-SEM da Intensidade Acústica

```

nomes_musicas <- colnames(dados_ia_pls)

# 1. Separar os itens pelos 4 quadrantes específicos
itens_al <- grep("AL_", nomes_musicas, value = TRUE)
itens_se <- grep("SE_", nomes_musicas, value = TRUE)
itens_mr <- grep("MR_", nomes_musicas, value = TRUE)
itens_tr <- grep("TR_", nomes_musicas, value = TRUE)

# 2. Estágio 1: Definir os 4 Módulos de IA
mm1_ia_4cat <- constructs(
  composite("IA_Alegria", itens_al, mode_A),
  composite("IA_Serenidade", itens_se, mode_A),
  composite("IA_Raiva", itens_mr, mode_A),
  composite("IA_Tristeza", itens_tr, mode_A)
)

# Relacionamento temporário para extrair scores
rs1_ia_4cat <- relationships(
  paths(from = c("IA_Alegria", "IA_Serenidade", "IA_Tristeza"), to = "IA_Raiva")
)

pls1_ia_4cat <- estimate_pls(dados_ia_pls, mm1_ia_4cat, rs1_ia_4cat)

```

```
## Generating the semnr model
```

```
## All 768 observations are valid.
```

```
summary(pls1_ia_4cat)
```

```

##
## Results from package semnr (2.3.7)
##
## Path Coefficients:
##           IA_Raiva
## R^2         0.917
## AdjR^2      0.917
## IA_Alegria   0.961

```



```
## IA_Serenidade    -0.343
## IA_Tristeza      0.339
##
## Reliability:
##               alpha rhoC  AVE rhoA
## IA_Alegria      0.991 0.992 0.923 0.991
## IA_Serenidade   0.990 0.991 0.935 0.990
## IA_Tristeza     0.992 0.993 0.933 0.992
## IA_Raiva        0.978 0.981 0.838 0.980
##
## Alpha, rhoC, and rhoA should exceed 0.7 while AVE should exceed 0.5
```

```
#plot(pls1_ia_4cat)
```

```
scores_ia_4cat <- as.data.frame(pls1_ia_4cat$construct_scores)

# 3. Estágio 2: Modelo Hierárquico de Intensidade (HMA) da IA
dados_ia_final_4cat <- cbind(scores_ia_4cat, ancora_t)

mm2_ia_4cat <- constructs(
  composite("Intensidade_IA",
    c("IA_Alegria", "IA_Serenidade", "IA_Raiva", "IA_Tristeza"),
    mode_A),
  composite("Âncora", colnames(ancora_t), mode_B)
)

rs2_ia_4cat <- relationships(
  paths(from = "Intensidade_IA", to = "Âncora")
)

pls2_ia_4cat <- estimate_pls(dados_ia_final_4cat, mm2_ia_4cat, rs2_ia_4cat)
```

```
## Generating the semnr model
```

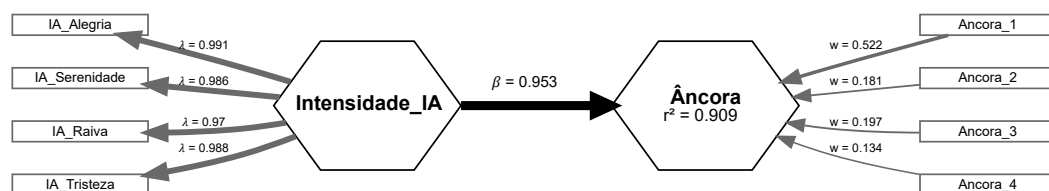
```
## All 768 observations are valid.
```

```
summary(pls2_ia_4cat)
```

```
##
## Results from package semnr (2.3.7)
##
## Path Coefficients:
##               Âncora
## R^2           0.909
## AdjR^2        0.908
## Intensidade_IA 0.953
##
## Reliability:
##               alpha rhoC  AVE rhoA
## Intensidade_IA 0.989 0.992 0.968 0.989
## Âncora         0.970 0.976 0.910 1.000
##
## Alpha, rhoC, and rhoA should exceed 0.7 while AVE should exceed 0.5
```

```
plot(pls2_ia_4cat)
```

```
## file:///C:/Users/vinic/AppData/Local/Temp/RtmpwR7EgA/fileddc5cfc701e/widgetddc3aa474c1.html screenshot
```



```
# 4. Bootstrap para verificar a significância dos pesos
boot_ia_4cat <- bootstrap_model(pls2_ia_4cat, nboot = 1000)
```

```
## Bootstrapping model using seminr...
```

```
## SEMinR Model successfully bootstrapped
```

```
summary(boot_ia_4cat)
```

```
##
```

```
## Results from Bootstrap resamples: 1000
```

```
##
```

```
## Bootstrapped Structural Paths:
```

##	Original Est.	Bootstrap Mean	Bootstrap SD	T Stat.	2.5% CI
##	0.953	0.952	0.012	76.714	0.923
##	97.5% CI				
##	0.972				

```
##
```

```
## Bootstrapped Weights:
```

##		Original Est.	Bootstrap Mean	Bootstrap SD
##	IA_Alegria -> Intensidade_IA	0.256	0.256	0.002
##	IA_Serenidade -> Intensidade_IA	0.253	0.253	0.002
##	IA_Raiva -> Intensidade_IA	0.254	0.254	0.002
##	IA_Tristeza -> Intensidade_IA	0.253	0.253	0.001
##	Ancora_1 -> Âncora	0.522	0.518	0.086
##	Ancora_2 -> Âncora	0.181	0.179	0.054
##	Ancora_3 -> Âncora	0.197	0.201	0.095
##	Ancora_4 -> Âncora	0.134	0.136	0.050

##		T Stat.	2.5% CI	97.5% CI
##	IA_Alegria -> Intensidade_IA	126.290	0.253	0.261
##	IA_Serenidade -> Intensidade_IA	168.376	0.251	0.257
##	IA_Raiva -> Intensidade_IA	125.602	0.251	0.259
##	IA_Tristeza -> Intensidade_IA	198.263	0.251	0.256
##	Ancora_1 -> Âncora	6.054	0.340	0.683
##	Ancora_2 -> Âncora	3.325	0.076	0.288
##	Ancora_3 -> Âncora	2.071	0.005	0.380
##	Ancora_4 -> Âncora	2.670	0.040	0.236

```
##
```

```
## Bootstrapped Loadings:
```

##		Original Est.	Bootstrap Mean	Bootstrap SD
##	IA_Alegria -> Intensidade_IA	0.991	0.990	0.002
##	IA_Serenidade -> Intensidade_IA	0.986	0.986	0.004
##	IA_Raiva -> Intensidade_IA	0.970	0.968	0.009
##	IA_Tristeza -> Intensidade_IA	0.988	0.987	0.004
##	Ancora_1 -> Âncora	0.988	0.987	0.005
##	Ancora_2 -> Âncora	0.933	0.930	0.018
##	Ancora_3 -> Âncora	0.972	0.970	0.009
##	Ancora_4 -> Âncora	0.922	0.917	0.022

##		T Stat.	2.5% CI	97.5% CI
##	IA_Alegria -> Intensidade_IA	402.030	0.985	0.994
##	IA_Serenidade -> Intensidade_IA	259.536	0.977	0.992
##	IA_Raiva -> Intensidade_IA	106.331	0.948	0.982
##	IA_Tristeza -> Intensidade_IA	274.010	0.979	0.992
##	Ancora_1 -> Âncora	205.763	0.975	0.994
##	Ancora_2 -> Âncora	52.466	0.891	0.960
##	Ancora_3 -> Âncora	103.153	0.947	0.984

```
## Ancora_4 -> Âncora          42.279    0.865    0.953
##
## Bootstrapped HTMT:
##   Original Est. Bootstrap Mean   Bootstrap SD       2.5% CI       97.5% CI
##         0.969         0.967         0.010         0.945         0.983
##
## Bootstrapped Total Paths:
##   Original Est. Bootstrap Mean   Bootstrap SD       2.5% CI       97.5% CI
##         0.953         0.952         0.012         0.923         0.972
```

4.2 Modelo Humano Hierárquico (PLS)

```
# 0. Itens selecionados pelo bootEGA
itens_c1 <- resultado_final %>% filter(Comunidade == 1) %>% pull(Item)
itens_c2 <- resultado_final %>% filter(Comunidade == 2) %>% pull(Item)
itens_c3 <- resultado_final %>% filter(Comunidade == 3) %>% pull(Item)
itens_c4 <- resultado_final %>% filter(Comunidade == 4) %>% pull(Item)

# 1. Garantir que a Acurácia esteja no mesmo dataframe
acc_hum <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>% # Filtra apenas os 38 estímulos validados
  group_by(Participants) %>%
  summarise(
    # Calcula a média da coluna Congruence (0 ou 1) para cada pessoa
    Acc = mean(as.numeric(Congruence), na.rm = TRUE)
  )

human_final_data <- human_wide_total %>%
  left_join(acc_hum, by = "Participants")

# 2. Estágio 1: Definir os 4 Módulos + Acurácia como âncora
mm1_hum <- constructs(
  composite("Alta_Ativação", itens_c1, mode_A),
  composite("Desprazer", itens_c2, mode_A),
  composite("Baixa_Ativação", itens_c3, mode_A),
  composite("Prazer", itens_c4, mode_A),
  composite("Acurácia", single_item("Acc")) # Âncora para extrair escores
)

# 3. Relacionamentos do Estágio 1 (Todos apontam para a Acurácia)
rs1_hum <- relationships(
  paths(from = c("Alta_Ativação", "Desprazer", "Baixa_Ativação", "Prazer"),
    to = "Acurácia")
)

# Estimar o modelo de primeiro estágio
pls1_hum <- estimate_pls(human_final_data, mm1_hum, rs1_hum)

## Generating the semirn model

## All 200 observations are valid.
```

```
summary(pls1_hum)
```

```
##
## Results from package semnr (2.3.7)
##
## Path Coefficients:
##           Acurácia
## R^2        0.119
## AdjR^2      0.101
## Alta_Ativação 0.162
## Desprazer   -0.168
## Baixa_Ativação 0.035
## Prazer      0.192
##
## Reliability:
##           alpha rhoC  AVE rhoA
## Alta_Ativação 0.936 0.941 0.485 0.946
## Desprazer     0.898 0.890 0.345 0.835
## Baixa_Ativação 0.949 0.953 0.515 0.966
## Prazer        0.862 0.878 0.377 0.820
## Acurácia      1.000 1.000 1.000 1.000
##
## Alpha, rhoC, and rhoA should exceed 0.7 while AVE should exceed 0.5
```

```
# Extrair os scores
scores_hum <- as.data.frame(pls1_hum$construct_scores)

# 3. Estágio 2: Formar a Intensidade Geral e prever Acurácia
# Juntar os scores com a acurácia
dados_hum_final <- cbind(scores_hum, Acc = human_final_data$Acc)

mm2_hum <- constructs(
  composite("Intensidade",
    c("Alta_Ativação", "Desprazer", "Baixa_Ativação", "Prazer"),
    mode_B),
  composite("Acurácia", single_item("Acc"))
)

rs2_hum <- relationships(
  paths(from = "Intensidade", to = "Acurácia")
)

pls2_hum <- estimate_pls(dados_hum_final, mm2_hum, rs2_hum)
```

```
## Generating the semnr model
```

```
## All 200 observations are valid.
```

```
summary(pls2_hum)
```

```
##
```

```
## Results from package seminr (2.3.7)
##
## Path Coefficients:
##           Acurácia
## R^2       0.119
## AdjR^2    0.115
## Intensidade 0.345
##
## Reliability:
##           alpha rhoC AVE rhoA
## Intensidade 0.601 0.508 0.380 1.000
## Acurácia    1.000 1.000 1.000 1.000
##
## Alpha, rhoC, and rhoA should exceed 0.7 while AVE should exceed 0.5
```

4. Bootstrap para verificar a Assimetria

```
boot_hum_final <- bootstrap_model(pls2_hum, nboot = 1000)
```

```
## Bootstrapping model using seminr...
```

```
## SEMinR Model successfully bootstrapped
```

```
summary(boot_hum_final)
```

```
##
## Results from Bootstrap resamples: 1000
##
## Bootstrapped Structural Paths:
## Original Est. Bootstrap Mean Bootstrap SD T Stat. 2.5% CI
##           0.345          0.363          0.084      4.119      0.231
##           97.5% CI
##           0.496
##
## Bootstrapped Weights:
## Original Est. Bootstrap Mean Bootstrap SD
## Alta_Ativação -> Intensidade      0.468      0.461      0.264
## Desprazer -> Intensidade      -0.487      -0.452      0.212
## Baixa_Ativação -> Intensidade      0.101      0.076      0.234
## Prazer -> Intensidade      0.556      0.510      0.207
## Acc -> Acurácia      1.000      1.000      0.000
## T Stat. 2.5% CI 97.5% CI
## Alta_Ativação -> Intensidade      1.772    -0.099      0.937
## Desprazer -> Intensidade      -2.292    -0.811     -0.061
## Baixa_Ativação -> Intensidade      0.430    -0.380      0.519
## Prazer -> Intensidade      2.683      0.084      0.854
## Acc -> Acurácia      .      1.000      1.000
##
## Bootstrapped Loadings:
## Original Est. Bootstrap Mean Bootstrap SD
## Alta_Ativação -> Intensidade      0.770      0.723      0.172
## Desprazer -> Intensidade      -0.396     -0.367      0.208
## Baixa_Ativação -> Intensidade      0.517      0.481      0.181
```

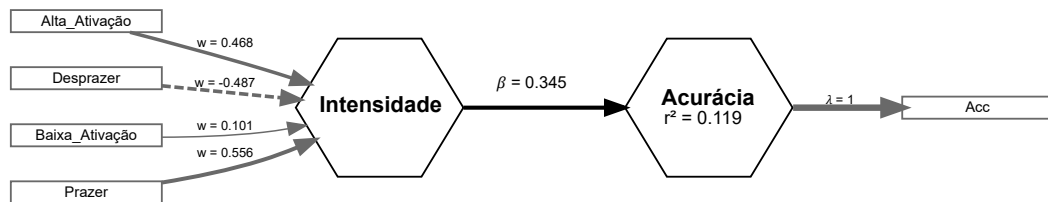
```
## Prazer -> Intensidade          0.710          0.658          0.163
## Acc -> Acurácia                1.000          1.000          0.000
##                               T Stat. 2.5% CI 97.5% CI
## Alta_Ativação -> Intensidade    4.464    0.333    0.955
## Desprazer -> Intensidade        -1.901   -0.726    0.012
## Baixa_Ativação -> Intensidade    2.851    0.089    0.778
## Prazer -> Intensidade            4.359    0.335    0.896
## Acc -> Acurácia                  .      1.000    1.000
##
## Bootstrapped HTMT:
## Original Est. Bootstrap Mean Bootstrap SD      2.5% CI      97.5% CI
##           0.392           0.389           0.087           0.221           0.557
##
## Bootstrapped Total Paths:
## Original Est. Bootstrap Mean Bootstrap SD      2.5% CI      97.5% CI
##           0.345           0.363           0.084           0.231           0.496
```

```
# 5. Resultados
summary(pls2_hum)
```

```
##
## Results from package semnr (2.3.7)
##
## Path Coefficients:
##           Acurácia
## R^2          0.119
## AdjR^2        0.115
## Intensidade   0.345
##
## Reliability:
##           alpha rhoC AVE rhoA
## Intensidade 0.601 0.508 0.380 1.000
## Acurácia    1.000 1.000 1.000 1.000
##
## Alpha, rhoC, and rhoA should exceed 0.7 while AVE should exceed 0.5
```

```
# Gráfico do Modelo Hierárquico Humano
plot(pls2_hum)
```

```
## file:///C:/Users/vinic/AppData/Local/Temp/RtmpwR7EgA/fileddc70897197/widgetddc4060998.html screenshot
```

5 Relações entre as variáveis

5.1 Diferenças entre o vetor positivo e o negativo

```
# --- PARTE 1: ORGANIZAR SCORES DA IA (38 Músicas) ---
# Extrair os scores do modelo de 4 quadrantes da IA
# 1. Pegar os quadrantes do Estágio 1
ia_quadrantes <- as.data.frame(pls1_ia_4cat$construct_scores)

# 2. Pegar a Intensidade do Estágio 2
ia_intensidade <- as.data.frame(pls2_ia_4cat$construct_scores)

# 3. Fundir tudo em um único dataframe de 38 músicas
scores_ia_4módulos <- ia_quadrantes %>%
  mutate(
    Intensidade_IA = ia_intensidade$Intensidade_IA,
    RowName = rownames(ia_quadrantes),
    Stimulus_ID = as.numeric(gsub("[^0-9]", "", RowName))
  ) %>%
  dplyr::select(Stimulus_ID, RowName, Intensidade_IA,
               IA_Alegria, IA_Serenidade, IA_Raiva, IA_Tristeza)

# --- PARTE 2: ORGANIZAR SCORES HUMANOS (N = 200/216) ---
# Assumindo que dados_hum_final já tem o Stimulus_ID e a Acurácia
# 1. Unificar os escores dos dois estágios dos Humanos (N = 200)
hum_st1 <- as.data.frame(pls1_hum$construct_scores)
hum_st2 <- as.data.frame(pls2_hum$construct_scores)

scores_hum_consolidado <- hum_st1 %>%
  mutate(
    Intensidade_Geral_Humana = hum_st2$Intensidade_Percebida,
    Participants = human_final_data$Participants
  )

print(">>> Escores Humanos por Participante consolidados <<<")

## [1] ">>> Escores Humanos por Participante consolidados <<<"

# 1. Gerar as médias humanas para cada uma das 38 músicas (do banco long)
hum_media_por_musica <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  group_by(Stimulus_ID) %>%
  summarise(
    Arousal_Humano = mean(Arousal, na.rm = TRUE),
    Valence_Humano = mean(Valence, na.rm = TRUE)
  )

# 2. Unir com os escores da IA (que já estão no nível de música)
df_correlacao_final <- inner_join(scores_ia_4módulos, hum_media_por_musica, by = "Stimulus_ID")

# 3. Teste de Correlação de Spearman (IA vs. Humano)
```

```
# Vamos ver como a Intensidade_IA se correlaciona com o Arousal e Valence humanos
cor_spearman <- cor(df_correlacao_final %>%
  dplyr::select(Intensidade_IA, IA_Alegria, IA_Serenidade, IA_Raiva, IA_Tristeza,
    Arousal_Humano, Valence_Humano),
  method = "spearman")

print(">>> MATRIZ DE CORRELAÇÃO IA vs. HUMANO (N=38) <<<")
```

```
## [1] ">>> MATRIZ DE CORRELAÇÃO IA vs. HUMANO (N=38) <<<"
```

```
print(round(cor_spearman, 3))
```

```
##           Intensidade_IA IA_Alegria IA_Serenidade IA_Raiva IA_Tristeza
## Intensidade_IA           1.000      0.963          0.960    0.908      0.965
## IA_Alegria                0.963      1.000          0.920    0.870      0.912
## IA_Serenidade             0.960      0.920          1.000    0.793      0.975
## IA_Raiva                  0.908      0.870          0.793    1.000      0.796
## IA_Tristeza               0.965      0.912          0.975    0.796      1.000
## Arousal_Humano            0.211      0.192          0.260    0.113      0.244
## Valence_Humano            0.003     -0.019         -0.058    0.035     -0.006
##           Arousal_Humano Valence_Humano
## Intensidade_IA           0.211          0.003
## IA_Alegria                0.192         -0.019
## IA_Serenidade             0.260         -0.058
## IA_Raiva                  0.113          0.035
## IA_Tristeza               0.244         -0.006
## Arousal_Humano            1.000          0.142
## Valence_Humano            0.142          1.000
```

```
# 1. Preparar o banco de dados longo para o modelo misto
df_misto_final <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  # Trazer a Intensidade da IA para cada linha de julgamento humano
  left_join(scores_ia_4módulos %>% dplyr::select(Stimulus_ID, Intensidade_IA), by = "Stimulus_ID")

# 2. Rodar o Modelo Misto (LMM)
# DV: Arousal Real (Humano)
# IV: Intensidade_IA (Física/MERT) multiplicada pela Categoria (Moderação)
library(lmerTest)
```

```
##
## Anexando pacote: 'lmerTest'

## O seguinte objeto é mascarado por 'package:lme4':
##
##      lmer

## O seguinte objeto é mascarado por 'package:stats':
##
##      step
```

```
modelo_paradoxo_lmm <- lmer(Arousal ~ Intensidade_IA * Intended_Emotion + (1|Participants),
                             data = df_misto_final)
```

```
# 3. RESULTADOS DO PARADOXO
```

```
print(">>> RESULTADOS DO MODELO MISTO: O PARADOXO <<<")
```

```
## [1] ">>> RESULTADOS DO MODELO MISTO: O PARADOXO <<<"
```

```
summary(modelo_paradoxo_lmm)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Arousal ~ Intensidade_IA * Intended_Emotion + (1 | Participants)
## Data: df_misto_final
##
## REML criterion at convergence: 29960.9
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -3.6931 -0.5888  0.0317  0.6177  3.3906
##
## Random effects:
## Groups      Name      Variance Std.Dev.
## Participants (Intercept) 1.756    1.325
## Residual              2.763    1.662
## Number of obs: 7600, groups: Participants, 200
##
## Fixed effects:
##                                Estimate Std. Error      df
## (Intercept)                   5.29537    0.10021 239.96006
## Intensidade_IA                 0.22678    0.02639 7393.00001
## Intended_EmotionHappiness      0.28213    0.06955 7393.00001
## Intended_EmotionSadness        0.06878    0.06378 7393.00001
## Intended_EmotionSerenity       0.29377    0.04783 7393.00001
## Intensidade_IA:Intended_EmotionHappiness 0.12394    0.11386 7393.00001
## Intensidade_IA:Intended_EmotionSadness  -0.07072    0.04672 7393.00001
## Intensidade_IA:Intended_EmotionSerenity  -0.49934    0.04159 7393.00001
##                                t value Pr(>|t|)
## (Intercept)                   52.845 < 2e-16 ***
## Intensidade_IA                 8.592 < 2e-16 ***
## Intended_EmotionHappiness      4.056 5.03e-05 ***
## Intended_EmotionSadness        1.079  0.281
## Intended_EmotionSerenity       6.142 8.56e-10 ***
## Intensidade_IA:Intended_EmotionHappiness  1.088  0.276
## Intensidade_IA:Intended_EmotionSadness  -1.514  0.130
## Intensidade_IA:Intended_EmotionSerenity -12.007 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##                                (Intr) Int_IA Int_EH Intndd_EmtnSd Intndd_EmtnSr
## Intensdd_IA                  -0.024
```

```
## Intndd_EmtH          -0.181  0.035
## Intndd_EmtnSd        -0.197  0.038  0.284
## Intndd_EmtnSr        -0.263  0.051  0.379  0.414
## Int_IA:I_EH          0.006 -0.232  0.146 -0.009      -0.012
## Intnsdd_IA:Intndd_EmtnSd 0.014 -0.565 -0.020  0.269      -0.029
## Intnsdd_IA:Intndd_EmtnSr 0.015 -0.635 -0.022 -0.024      -0.268
##                      I_IA:I_EH Intnsdd_IA:Intndd_EmtnSd
## Intnsdd_IA
## Intndd_EmtH
## Intndd_EmtnSd
## Intndd_EmtnSr
## Int_IA:I_EH
## Intnsdd_IA:Intndd_EmtnSd 0.131
## Intnsdd_IA:Intndd_EmtnSr 0.147      0.359
```

```
# 4. CAPACIDADE EXPLICATIVA (R2)
performance::r2(modelo_paradoxo_lmm)
```

```
## # R2 for Mixed Models
##
## Conditional R2: 0.398
## Marginal R2: 0.016
```

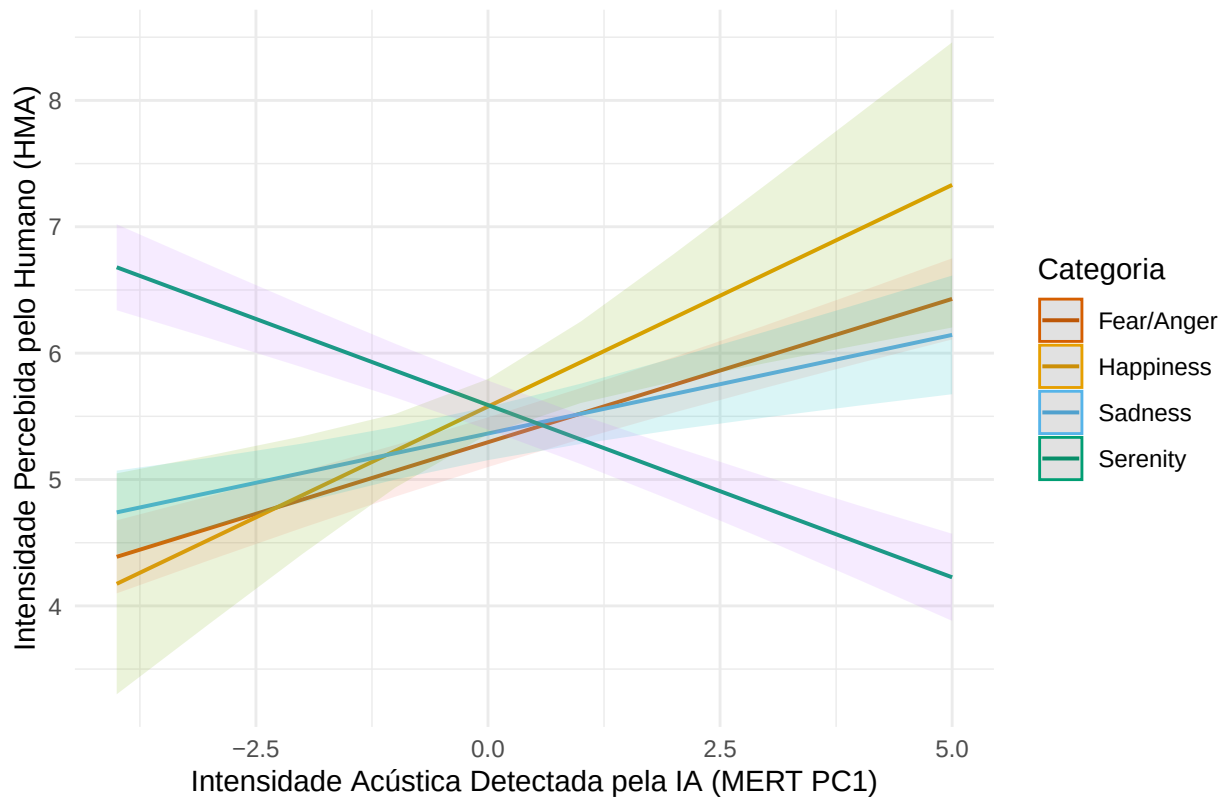
```
library(ggeffects)

# Gerar as predições do modelo
pred_paradoxo <- predict_response(modelo_paradoxo_lmm,
                                   terms = c("Intensidade_IA", "Intended_Emotion"))

# Plotar com as cores padronizadas
plot(pred_paradoxo) +
  scale_color_manual(values = c("Fear/Anger" = "#D55E00", "Happiness" = "#E69F00",
                                "Sadness" = "#56B4E9", "Serenity" = "#009E73")) +
  labs(
    title = "O Paradoxo da Serenidade: Falha na Linearidade da IA",
    x = "Intensidade Acústica Detectada pela IA (MERT PC1)",
    y = "Intensidade Percebida pelo Humano (HMA)",
    color = "Categoria"
  ) +
  theme_minimal()
```

```
## Scale for colour is already present.
## Adding another scale for colour, which will replace the existing scale.
```

O Paradoxo da Serenidade: Falha na Linearidade da IA



```
# 1. Pegar os escores latentes humanos e associar às músicas
# Usaremos a lógica de pesos que o seu PLS-SEM validou
# Intensidade Humana = (Arousal * 0,61) + (Valence * 0,32) [conforme sua PCA/PLS]

df_song_level_human <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  mutate(Intensidade_Humana_Latente = (Arousal * 0.61) + (Valence * 0.32)) %>%
  group_by(Stimulus_ID) %>%
  summarise(
    Intensidade_Hum_Média = mean(Intensidade_Humana_Latente),
    # Médias dos escores brutos para controle
    Arousal_Médio = mean(Arousal),
    Valence_Médio = mean(Valence)
  )

# 2. Unir com os escores latentes da IA
df_confluencia_latente <- inner_join(scores_ia_4módulos, df_song_level_human, by = "Stimulus_ID")

# Matriz de correlação de Spearman entre os "Cérebros"
cor_latente <- cor(df_confluencia_latente %>%
  dplyr::select(Intensidade_IA, IA_Alegria, IA_Serenidade, IA_Raiva, IA_Tristeza,
    Intensidade_Hum_Média, Arousal_Médio, Valence_Médio),
  method = "spearman")

print(">>> CONFLUÊNCIA ENTRE VARIÁVEIS LATENTES (IA vs. HUMANO) <<<")
```

```
## [1] ">>> CONFLUÊNCIA ENTRE VARIÁVEIS LATENTES (IA vs. HUMANO) <<<"
```

```
print(round(cor_latente[1:5, 6:8], 3))
```

```
##               Intensidade_Hum_Média Arousal_Médio Valence_Médio
## Intensidade_IA              0.159          0.211          0.003
## IA_Alegria                  0.137          0.192         -0.019
## IA_Serenidade               0.150          0.260         -0.058
## IA_Raiva                     0.117          0.113          0.035
## IA_Tristeza                 0.171          0.244         -0.006
```

```
# --- 1. EXTRAIR A INTENSIDADE LATENTE DA IA (PC1) ---
# Usamos o componente unipolar massivo (87% de variância)
df_ia_loadings <- data.frame(
  Stimulus_ID = as.numeric(gsub("[^0-9]", "", rownames(pca_ia$rotation))),
  IA_Magnitude_Latente = pca_ia$rotation[,1]
)
# Garantir sinal positivo para Magnitude
if(mean(df_ia_loadings$IA_Magnitude_Latente) < 0) {
  df_ia_loadings$IA_Magnitude_Latente <- df_ia_loadings$IA_Magnitude_Latente * -1
}

# --- 2. EXTRAIR A INTENSIDADE LATENTE HUMANA (PC1 das Médias) ---
# Primeiro, criamos uma matriz de médias (N=38 músicas x 2 dimensões brutas)
df_human_averages <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  group_by(Stimulus_ID) %>%
  summarise(M_Arousal = mean(Arousal), M_Valence = mean(Valence)) %>%
  column_to_rownames("Stimulus_ID")

# Rodamos um PCA de fator único sobre as médias humanas
pca_hum_song <- principal(df_human_averages, nfactors = 1)

df_hum_loadings <- data.frame(
  Stimulus_ID = as.numeric(rownames(pca_hum_song$scores)),
  Humano_Intensidade_Latente = as.numeric(pca_hum_song$scores[,1])
)

# --- 3. O CONFRONTO DE ESTIMATIVAS ---
df_convergencia_final <- inner_join(df_ia_loadings, df_hum_loadings, by = "Stimulus_ID") %>%
  left_join(gabarito_categorias, by = "Stimulus_ID")

# Teste de Spearman (Invariante à escala bruta)
cor_final <- cor.test(df_convergencia_final$IA_Magnitude_Latente,
  df_convergencia_final$Humano_Intensidade_Latente,
  method = "spearman")

print(">>> CONVERGÊNCIA ESTRUTURAL: IA (PC1) vs. HUMANO (PC1) <<<")
```

```
## [1] ">>> CONVERGÊNCIA ESTRUTURAL: IA (PC1) vs. HUMANO (PC1) <<<"
```

```
print(cor_final)
```

```
##
## Spearman's rank correlation rho
##
## data: df_convergencia_final$IA_Magnitude_Latente and df_convergencia_final$Humano_Intensidade_Latente
## S = 7716, p-value = 0.3492
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## 0.1557063
```

```
# 1. Gerar as médias de Arousal e Valence por música (se ainda não tiver o objeto)
```

```
hum_medias_sam <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  group_by(Stimulus_ID) %>%
  summarise(
    Arousal_Humano = mean(Arousal, na.rm = TRUE),
    Valence_Humano = mean(Valence, na.rm = TRUE)
  )
```

```
# 2. Unir com o seu dataframe de convergência
```

```
df_convergencia_final <- df_convergencia_final %>%
  left_join(hum_medias_sam, by = "Stimulus_ID")
```

```
# 3. Calcular a Distância Euclidiana (Raiz da soma dos quadrados dos desvios do centro 5)
```

```
df_convergencia_final <- df_convergencia_final %>%
  mutate(
    Distancia_Euclidiana_Humana = sqrt((Arousal_Humano - 5)^2 + (Valence_Humano - 5)^2)
  )
```

```
# 4. Rodar a regressão: A magnitude da IA explica a geometria humana?
```

```
mod_euclidiano <- lm(Distancia_Euclidiana_Humana ~ IA_Magnitude_Latente, data = df_convergencia_final)
```

```
# 5. RESULTADOS PARA O TEXTO
```

```
summary(mod_euclidiano)
```

```
##
## Call:
## lm(formula = Distancia_Euclidiana_Humana ~ IA_Magnitude_Latente,
##     data = df_convergencia_final)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.78242 -0.43884 -0.05888  0.32160  1.28929
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      3.858      2.052   1.880  0.0683 .
## IA_Magnitude_Latente -17.899     12.651  -1.415  0.1657
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```



```
##
## Residual standard error: 0.5253 on 36 degrees of freedom
## Multiple R-squared: 0.05267, Adjusted R-squared: 0.02636
## F-statistic: 2.002 on 1 and 36 DF, p-value: 0.1657

cat("\n>>> R2 do Modelo Geométrico (Distância):", round(summary(mod_euclidiano)$r.squared, 4))

##
## >>> R2 do Modelo Geométrico (Distância): 0.0527
```

6 Russell vs HMA

```
# =====
# O DUELO DEFINITIVO DE ACURÁCIA (RUSSELL vs MÁQUINA/HMA) - ESTUDO 1
# =====

library(lme4)
library(performance)
library(dplyr)
library(stringr)

# 1. Extrair os 3 primeiros PCs da IA (MERT) a partir do PCA que você já rodou
# Como você usou prcomp em uma matriz transposta, os scores das músicas estão em "rotation"
df_pcs_ia <- data.frame(
  # Extrai apenas os números (IDs) dos nomes das linhas (ex: "AL_4" vira 4)
  Stimulus_ID = as.numeric(str_extract(rownames(pca_ia$rotation), "\\d+")),
  PC1_MERT     = as.numeric(pca_ia$rotation[, 1]),
  PC2_MERT     = as.numeric(pca_ia$rotation[, 2]),
  PC3_MERT     = as.numeric(pca_ia$rotation[, 3])
)

# Garantir a unipolaridade do PC1 (se a média estiver invertida, consertamos)
if(mean(df_pcs_ia$PC1_MERT) < 0) df_pcs_ia$PC1_MERT <- df_pcs_ia$PC1_MERT * -1

# 2. Preparar os dados humanos (data_long já existe no seu environment)
df_duelo_acc <- data_long %>%
  filter(Stimulus_ID %in% ids_38) %>%
  mutate(
    # Garantir formato numérico
    Valence = as.numeric(Valence),
    Arousal = as.numeric(Arousal),
    Congruence_Num = as.numeric(Congruence)
  ) %>%
  # Limpar possíveis NAs gerados
  filter(!is.na(Valence), !is.na(Arousal), !is.na(Congruence_Num)) %>%

# A lógica cirúrgica de Russell: Ipsatizar POR PARTICIPANTE
group_by(Participants) %>%
  mutate(
    # scale(center = T, scale = T) centraliza na média e divide pelo DP do indivíduo
    Valence_Ipsat = scale(Valence, center = TRUE, scale = TRUE),
    Arousal_Ipsat = scale(Arousal, center = TRUE, scale = TRUE)
```

```

) %>%
ungroup() %>%

# Cruzar com os dados da IA
left_join(df_pcs_ia, by = "Stimulus_ID")

# 3. O DUELO DE MODELOS MISTOS BINOMIAIS (glmer)
cat("\n>>> ESTIMANDO MODELO DE RUSSELL (Aguarde...) <<<\n")

##
## >>> ESTIMANDO MODELO DE RUSSELL (Aguarde...) <<<

mod_russell_acc <- glmer(Congruence_Num ~ Valence_Ipsat + Arousal_Ipsat + (1 | Participants),
                        data = df_duelo_acc,
                        family = binomial,
                        control = glmerControl(optimizer = "bobyqa"))

cat("\n>>> ESTIMANDO MODELO HMA/MÁQUINA (Aguarde...) <<<\n")

##
## >>> ESTIMANDO MODELO HMA/MÁQUINA (Aguarde...) <<<

mod_hma_acc <- glmer(Congruence_Num ~ PC1_MERT + PC2_MERT + PC3_MERT + (1 | Participants),
                    data = df_duelo_acc,
                    family = binomial,
                    control = glmerControl(optimizer = "bobyqa"))

# 4. O VEREDITO DE PREDIÇÃO DE ACURÁCIA
cat("\n>>> RESULTADO FINAL DO DUELO: RUSSELL vs HMA <<<\n")

##
## >>> RESULTADO FINAL DO DUELO: RUSSELL vs HMA <<<

veredito_acc <- compare_performance(mod_russell_acc, mod_hma_acc, metrics = c("AIC", "R2", "RMSE"))
print(veredito_acc)

## # Comparison of Model Performance Indices
##
## Name | Model | AIC (weights) | R2 (cond.) | R2 (marg.) | RMSE
## -----
## mod_russell_acc | glmerMod | 9315.7 (<.001) | 0.130 | 0.029 | 0.448
## mod_hma_acc | glmerMod | 9179.3 (>.999) | 0.161 | 0.060 | 0.443

sessionInfo()

## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default

```

```

## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8 LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] lmerTest_3.1-3      rstatix_0.7.3      ggeffects_2.3.1    gridExtra_2.3
## [5] tibble_3.3.1        stringr_1.6.0      boot_1.3-32        seminr_2.3.7
## [9] psych_2.5.3         sjPlot_2.9.0       lme4_1.1-37        Matrix_1.7-3
## [13] EGAnet_2.3.0        performance_0.15.2 ggpubr_0.6.2       readxl_1.4.5
## [17] ggrepel_0.9.6       ggplot2_4.0.2      tidyr_1.3.1        dplyr_1.1.4
## [21] pacman_0.5.1
##
## loaded via a namespace (and not attached):
## [1] libcoin_1.0-10      RColorBrewer_1.1-3  jsonlite_2.0.0
## [4] rstudioapi_0.17.1   datawizard_1.3.0    magrittr_2.0.4
## [7] TH.data_1.1-5       modeltools_0.2-24   farver_2.1.2
## [10] nloptr_2.2.1        rmarkdown_2.30      vctrs_0.6.5
## [13] minqa_1.2.8         base64enc_0.1-3     webshot_0.5.5
## [16] forcats_1.0.1       htmltools_0.5.8.1   progress_1.2.3
## [19] haven_2.5.5         broom_1.0.10        cellranger_1.1.0
## [22] Formula_1.2-5       parallelly_1.44.0   htmlwidgets_1.6.4
## [25] plyr_1.8.9          sandwich_3.1-1      zoo_1.8-14
## [28] igraph_2.2.1        lifecycle_1.0.5     pkgconfig_2.0.3
## [31] sjlabelled_1.2.0    webshot2_0.1.2      R6_2.6.1
## [34] fastmap_1.2.0       snakecase_0.11.1    rbibutils_2.3
## [37] future_1.68.0       numDeriv_2016.8-1.1 digest_0.6.38
## [40] fdrtool_1.2.18      colorspace_2.1-1    GGally_2.4.0
## [43] ps_1.9.1            Hmisc_5.2-5         labeling_0.4.3
## [46] progressr_0.18.0    abind_1.4-8         compiler_4.5.0
## [49] withr_3.0.2         glasso_1.11         htmlTable_2.4.3
## [52] S7_0.2.1           backports_1.5.0     carData_3.0-5
## [55] ggstats_0.11.0      ggsignif_0.6.4      MASS_7.3-65
## [58] corpcor_1.6.10      gtools_3.9.5        chromote_0.5.1
## [61] tools_4.5.0         pbivnorm_0.6.0      foreign_0.8-90
## [64] future.apply_1.20.0 nnet_7.3-20         glue_1.8.0
## [67] quadprog_1.5-8      DiagrammeR_1.0.11   promises_1.3.2
## [70] nlme_3.1-168        grid_4.5.0          checkmate_2.3.2
## [73] cluster_2.1.8.1     reshape2_1.4.5      generics_0.1.4
## [76] gtable_0.3.6        websocket_1.4.4      sna_2.8
## [79] data.table_1.17.0   hms_1.1.4           car_3.1-3
## [82] coin_1.4-3          utf8_1.2.6          pillar_1.11.1
## [85] later_1.4.2         splines_4.5.0       lattice_0.22-7
## [88] survival_3.8-3      tidysselect_1.2.1   pbapply_1.7-4
## [91] knitr_1.50          reformulas_0.4.2    stats4_4.5.0
## [94] xfun_0.52           qgraph_1.9.8        matrixStats_1.5.0

```

## [97] visNetwork_2.1.4	stringi_1.8.7	statnet.common_4.11.0
## [100] yaml_2.3.10	evaluate_1.0.5	codetools_0.2-20
## [103] cli_3.6.5	rpart_4.1.24	Rdpack_2.6.4
## [106] processx_3.8.6	lavaan_0.6-20	network_1.19.0
## [109] Rcpp_1.1.0	globals_0.18.0	coda_0.19-4.1
## [112] png_0.1-8	parallel_4.5.0	prettyunits_1.2.0
## [115] jpeg_0.1-11	listenv_0.10.0	mvtnorm_1.3-3
## [118] scales_1.4.0	insight_1.4.2	purrr_1.2.1
## [121] crayon_1.5.3	rlang_1.1.6	cowplot_1.2.0
## [124] multcomp_1.4-29	mnormt_2.1.1	